目录

微处理器接口 2
9.1 多路复用 7 段 LED 展示 3
9.1.1 工作原理 3
9.1.2 控制器设计 4
9.2 连接开关的问题 6
9.3 3×4 键盘控制器 11
9.3.1 工作原理 12
9.3.2 控制器设计 13
9.4 PS2 键盘和鼠标 15
9.4.1 工作原理—PS2 键盘 15
9.4.2 控制器设计—PS2 键盘 16
9.4.3 工作原理—PS2 键盘 20
9.4.4 控制器设计—PS2 键盘 21
9.5 RS-232 蓝牙通信控制器 26
9.5.1 RS-232 工作原理 27
9.5.2 RS-232 控制器设计 27
9.5.3 应用实现 29
9.6 液晶显示控制器 30
9.6.1 操作原理 30
9.6.2 控制器设计 32
9.7 VGA 显示器控制器 36
9.7.1 工作原理 36
9.7.2 控制器设计 38
9.7.3 控制器实现 43
9.8 用于温度传感器的 A/D 控制器 44
9.8.1 工作原理 44
9.8.2 控制器设计 45
9.8.3 应用实现 49
9.9 实时时钟的 I^2C 总线控制器 50
9.9.1 工作原理 50
9.9.2 控制器设计 52
9.9.3 应用实现 56
9.10 USB 的 SPI 总线控制器 56
9.10.1 操作原理 – SPI 57
9.10.2 操作原理 – USB 57
9.11 习题 59

第九章

微处理器接口

对微处理器而言，具备一个能在微处理器与人之间、电子设备与现实世界间交互接口是非常有用的。在现实世界中得到的测量值通常是模拟的，即是一些连续值。然而在微处理器中存储的信息确实离散的数字值。并且对微处理器而言，其与其他电子元件的通信必须遵循设备间的标准通信协议。在这一章中，我们将介绍一些通用的 I/O 接口及标准通信协议及微处理器是如何与其他电子元件连接的。

9.1 多路复用 7 段 LED 展示

目前为止我们已经完成了离散 LED 灯及单个 7 段 LED 灯的展示。为了显示更大的十进制数字，通常两个或者更多的 7 段 LED 灯将会结合使用，如 9.1 所示。



图 9.1 一个 8 位的 7 段显示器，显示数字 3.1415

9.1.1 工作原理

我们知道每个 7 段 LED 灯需要 7 个 LED 灯连接（不包括小数点），因此如果要展示 8 个 7 段 LED 灯，则需要 56 个连接。为了减少所需连接数，这 8 个 7 段 LED 灯最好能以某种方式共同连接。在内部，每个 7 段 LED 灯相同的段将连接在一起。换句话说，所有 7 段 LED 灯的 a 段将连接在一起，所有 7 段 LED 灯的 b 段将连接在一起,等等。这样做的好处在于，无论有多少个 7 段 LED 灯，总是只需要 7 个连接。而且，对于每个 7 段 LED 灯他们将共用一个阳极或者阴极。

图 9.2 展示了一个由 3 个共用阴极的 7 段 LED 灯的内在连接图。对于一个共用阴极的展示，当数字连接端置 1 时，整个对应数字关闭。当数字连接端置 0 且段连接端置 1 时该特定 LED 打开。因为对所有的 7 段 LED 灯而言，相同的段都是连接在一起的，因此，如果你连接了，即段 a 置 1 和所有的三个 7 段 LED 灯置 0，则所有的三段 7 段 LED 灯都将打开。那么现在的问题在于如何打开某个特定 7 段 LED 灯的某段而关闭其他 7 段 LED 灯的该段。举个例子，为了展示十进制数字 45，我们需要关闭第 1 个 7 段 LED 灯的段 a 并且打开第 0 个 7 段 LED 灯的段 a.但是为了打开着两个 7 段 LED 灯，他们的连接必须连到 0。但是如果它们都连接到 0，那么它们 a 段将都会一起打开或者关闭。

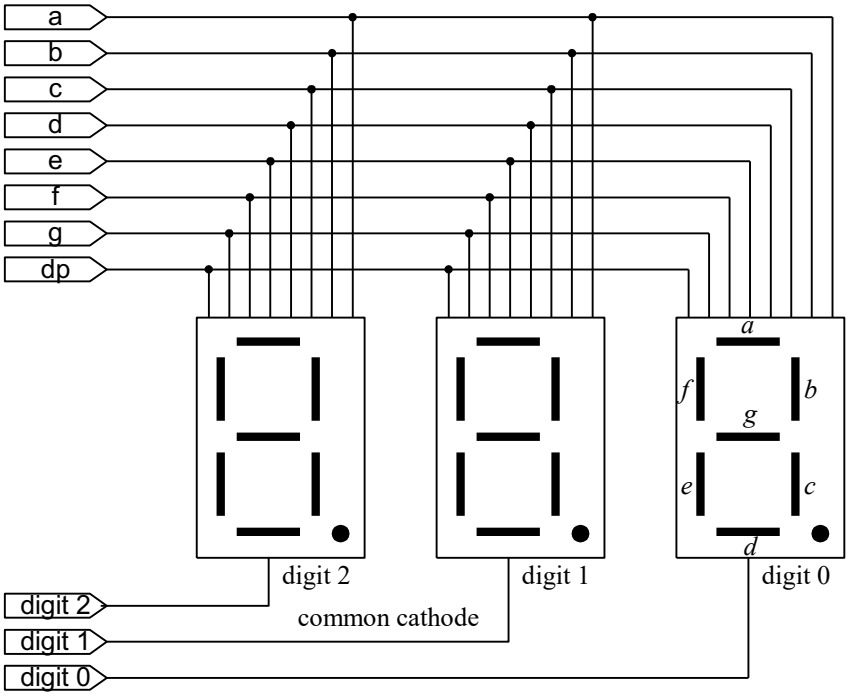


图 9.2 共极的 3 位 7 段显示器的内部连接

该问题的解决方案就是时间复用。时间复用将在某个非常短的时间只打开一个 7 段 LED 灯。在整个联系循环中，对于所有的 7 段 LED 灯，每次都将只打开一个。但是虽然这些灯只在某个时间打开，但我们还会有感觉他们总是在同一时间打开的，因为循环时间太短了。这样的代价就是尽管他们是连续，但亮度会略有改变。

9.1.2 控制器设计

状态图如 9.3 所示，对于一个 3 数字 7 段 LED 控制器，展示十进制数字 123 是极其简单的。它连续地在三个状态中循环即可。

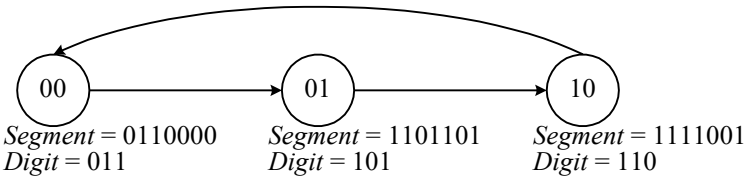


图 9.3 显示数字 123 的 3 位 7 段显示器的状态图

因为 7 段 LED 的复用，主要考虑的问题便是如何使得 FSM 循环 快速的在三个状态中循环。如果时钟速度太低，那么你将在某个时刻只看到一个 7 段 LED 亮了。当你渐渐的提高时钟速度，你将看到所有的 3 个 7 段 LED 都会亮但是会有闪烁。继续增加时钟速度你将看到闪烁消失，它们一直亮着。如果你继续增加时钟速度，那么这些 7 段 LED 将开始变得有些亮度减弱。图 9.4 和 9.5 分别展示了这个用于显示数字 123 的 3 个 7 段 LED 灯的 Verilog 和 VHDL 代码。

```
module fsm (
  input Clock, Reset,
  output reg [2:0] Digit,
  output reg [0:6] Segment
    );

  reg [1:0] state;

  always @ (posedge Clock, posedge Reset) begin
    if (Reset) begin
      Digit <= 3'b111;         // off
      Segment <= 7'b0000000;   // off
      state<=2'b00;
      end
    else
      case (state)
      2'b00: begin
        // display the number 1 on digit 2
        Segment <= 7'b0110000; // 1
        Digit <= 3'b011;       // D2
        state <= 2'b01;
        end
      2'b01: begin
        // display the number 2 on digit 1
        Segment <= 7'b1101101; // 2
        Digit <= 3'b101;       // D1
        state<=2'b10;
        end
      2'b10: begin
        // display the number 3 on digit 0
        Segment <= 7'b1111001; // 3
```

```
       Digit <= 3'b110;       // D0
       state<=2'b00;
       end
     default: begin
       state<=2'b00;
       end
     endcase
  end // always
endmodule
```

图 9.4 3 位 7 段显示器控制器的行为级 Verilog 代码

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY fsm IS PORT (
  Clock, Reset: IN STD_LOGIC;
  Digit: OUT  STD_LOGIC_VECTOR (2 DOWNTO 0);
  Segment: OUT  STD_LOGIC_VECTOR (0 TO 6));
END fsm;

ARCHITECTURE Behavioral OF fsm IS
  SIGNAL state: STD_LOGIC_VECTOR(1 DOWNTO 0);
BEGIN
    PROCESS(Clock, Reset)
    BEGIN
    IF(Reset = '1') THEN
      Digit <= "111";
      Segment <= "0000000";
      state <= "00";
    ELSIF(Clock'EVENT AND Clock = '1') THEN
      CASE state IS
      WHEN "00" =>
        -- display the number 1 on digit 2
        Segment <= "0110000";  -- 1
        Digit <= "011";        -- D2
        state <= "01";
      WHEN "01" =>
        -- display the number 2 on digit 1
        Segment <= "1101101";  -- 2
        Digit <= "101";        -- D1
        state <= "10";
      WHEN "10" =>
        -- display the number 3 on digit 0
        Segment <= "1111001";  -- 3
        Digit <= "110";        -- D0
        state <= "00";
      WHEN OTHERS =>
        state <= "00";
      END CASE;
    END IF;
  END PROCESS;
END Behavioral;
```

图 9.5 3 位 7 段显示器控制器的行为级 VHDL 代码

9.2 连接开关的问题

开关是一种普通而又简单的输入设备，并且很容易和微处理器接口相连。但是，如果控制单元中的时序不正确，即使是这些简单的输入设备也无法发挥它预期的功能。在本节中，我们将列举这些开关在连接使用时可能会出现的一些潜在问题。

为了说明开关在连接使用时存在的问题，我们将设计一个专用的微处理器。通过一个输入端口输入多个 8 位无符号数字，然后输出这些数字的总和。只要输入的数字不是 0，该算法就继续输入数字。输入的每个数字会在输出中显示出来。当输入的数字是 0 时，则该算法停止，并输出所有输入数字的总和。用于解决该问题的算法如图 9.6 中所示。

```
1  sum = 0
2  BEGIN LOOP
3    INPUT X
4    sum = sum + X
5    IF (X = 0) THEN
6      EXIT LOOP
7    END IF
8    OUTPUT X
9  END LOOP
10   OUTPUT sum
```

图 9.6 所有输入数字求和问题的算法

图 9.6 中的算法有五个数据操作语句，分别在第 1,3,4,8 和第 10 行。第 5 行是一个条件判断语句。同时，该算法需要一个加法器和两个 8 位寄存器：一个用于变量 X，另一个用于变量总和。专用的数据通道如图 9.7 所示。

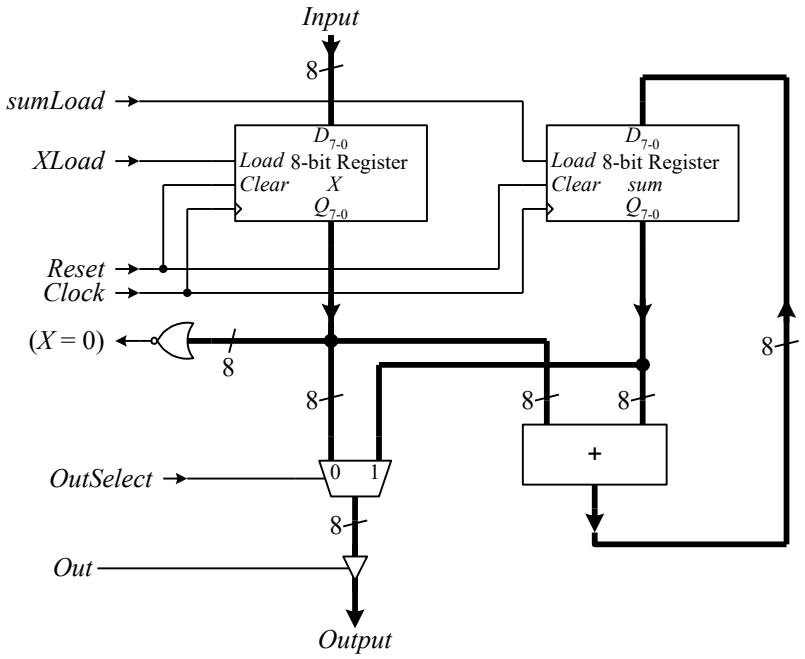


图 9.7 为解决所有输入数字求和问题的专用数据通道。

该算法的第 1 行受控于复位信号，在初始化时，让两个寄存器初始化为零，并且不需要对这一状态提供控制字。 第 3 行受控于 XLoad 信号。加法器的输入操作数来自两个寄存器，第 4 行受控于 sumLoad 信号。

信号 OutSelect 选择 2 至 1 多路复用器用于两个源：寄存器 X 或寄存器 sum，中的一个输出。控制信号 Out 用于使能所选中源的输出值的三态缓冲器。条件测试（X = 0）由被连接到 X 寄存器输出端的 8 输入与非门门产生。

这个用于解决输入数字求和问题的专用数据通路需要四个控制信号，sumLoad，XLoad，OutSelect，和 Out，以及所产生的一个状态信号（X=0）。控制字如图 9.8 所示。

控制字	指令	*XLoad*	*sumLoad*	*OutSelect*	*Out*
0	INPUT *X*	1	0	×	0
1	*sum = sum + X*	0	1	×	0
2	OUTPUT *X*	0	0	0	1
3	OUTPUT *sum*	0	0	1	1

图 9.8 用于解决输入数字求和问题的控制字

乍一看，这种算法似乎很简单很直接。然而，由于这一问题的要求，这种微处理器的实际硬件实现稍微有点困难。具体而言，对于多个不同的数字通过一个输入端口输入的要求，需要仔细的考虑时序和理解机械开关的行为。此外，对于输出寄存器 X 的值的时序更要认真思考。

作为第一次尝试，我们先从图 9.9（a）中所示的状态图开始。该算法的第 1 行由异步复位 Reset 执行，因此它不要求执行的状态。第 3 行执行状态 00，然后无条件地跟着执行第 4 行的状态 01。 之后条件（X =0）被检验。如果该条件为真，则退出循环，FSM 进入状态 11 输出总和 Sum 的值，并保持在这个状态直到检测到复位信号。如果条件为假，则 FSM 转到状态 10 输出 X，并重复该循环回到状态 00。

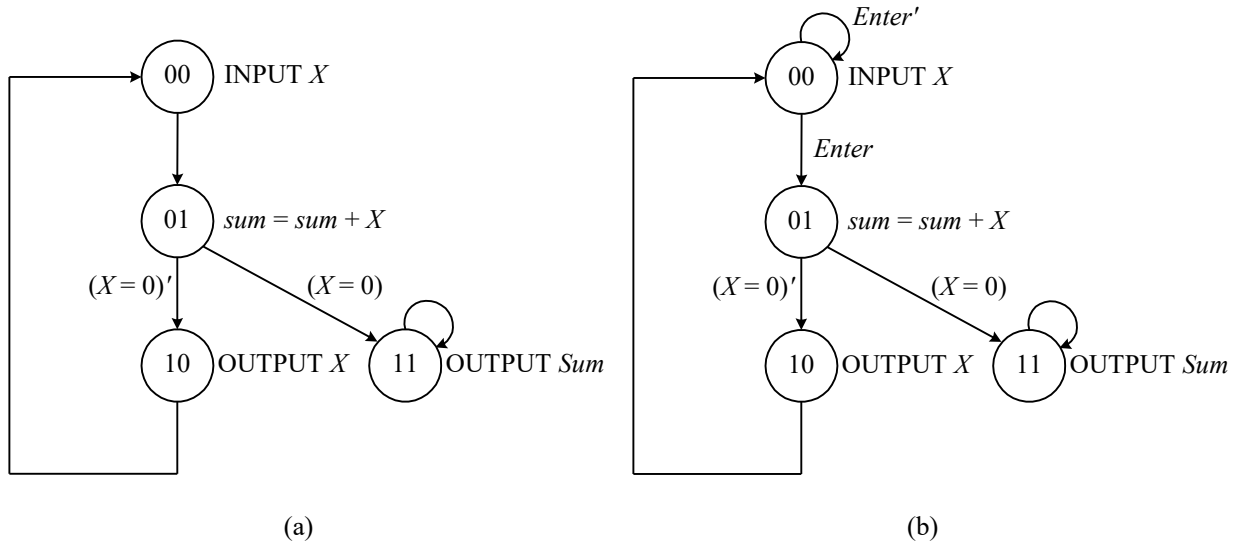


图 9.9 求解输入数字求和问题的不正确状态图

但是，如果在硬件中实现这个电路，它将无法正常工作。其原因是，因为该时钟速率非常快，通过三个循环状态（00，01，和 10）的 FSM 周期也执行的非常快。这样一来，FSM 将通过状态 00 输入一个数字很多次，直到输入另一个不同的数字。因此，相同的一个数字将被相加多次。

要解决这个问题，我们需要添加另一个类似于输入开关的输入信号。这样，FSM 将停留在状态 00，等待输入开关信号 Enter 被声明。这会给用户在按下输入开关信号 Enter 之前提供设置输入数字的时间。当 Enter 信号被声明时，FSM 将退出状态 00，用于处理新输入的数字。修改的状态图如图 9.9（b）中所示。

在修改的状态图中，还是会存在由于高速时钟所造成的时序问题。在按下 Enter 开关信号之后，在释放它之前存在一段时间，FSM 将再次拥有足够的时钟周期来完成整个循环并回到状态 00。但是，因为你还没有释放 Enter 开关，FSM 将以相同的输入数字继续进行到另一个完整的循环中去。要解决这个问题，我们需要打破这个循环，以等待 Enter 开关被释放。此时状态图如图 9.10（a）中所示。状态 10 在进入循环回到状态 00 之前需要等待 Enter 开关被释放。

从理论上讲，最后的状态图是正确的。然而，在实践中仍有可能存在由于使用输入开关 Enter 信号的机械开关所造成的操作方面的问题。当按下一个机械开关时，它通常在位置沉降下来之前进行多次的推移和关闭。这就是所谓的“反跳”问题。当开关被按下后，在开和关的位置之间有机械波动的时间里，FSM 可以再次经过多次循环。我们需要做的是去抖开关。然而，这不是在 FSM 电路本身进行，而是在 FSM 和开关按键之间的接口电路中实现的。我们回过头来在所建立的接口电路中解决这个问题。

现在，基于如图 9.10(a)中所示的状态图，我们先构建一个控制单元电路。四个状态被用于处理五个数据操作语句。除了状态 11 之外的所有状态，将输出 X。状态 00 输入 X，然后等待输入 Enter 信号。这使得用户能够设置所输入的数字，并按下 Enter 信号。当输入开关 Enter 信号被按压时，FSM 进入状态 01，求和 X，并判断条件（X =0）。如果条件为真，则 FSM 进入状态 11 并终止，输出最终的结果求和 Sum 信号；否则，它将进入状态 10，等待被用户所释放的 Enter 开关输入信号。之后会退出状态 10，FSM 继续在重复循环状态 00。

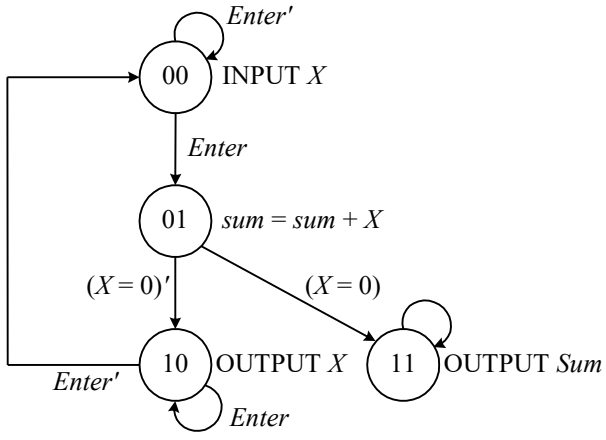
从状态图中导出的下一状态表，如图 9.10（b）中所示。该表需要四个变量：用于表示 4 个状态的两位数编码：Q1 和 Q0，以及两个状态信号：Enter 和 (X=0)。

使用 D 触发器来实现的状态存储，其执行表和下一个状态表是相同的，除了表中输入到触发器的值：D1 和 D0，代替触发器的输出：Q1next 和 Q0next。K-maps 和 Q1next 以及 Q0next 所表示的下一状态方程如图 9.10（c）中所示。

三个控制信号的控制字和输出表如图 9.10（d）中所示。状态 00 通过声明 XLoad 来执行图 9.6 中算法的第 3 行，然后通过解除声明 Out 信号来执行第 8 行。当 Out 解除声明后，X 被传递到输出。状态 01 执行第 4 行和第 8 行。第 4 行通过声明 sumLoad 信号来执行，第 8 号通过声明 Out 信号来执行。状态 10 再次通过声明 Out 信号来执第 8 行。最后，状态 11 通过声明 Out 信号来执行 10。

输出方程，从输出表中得到，如图 9.10（e）所示。这是一个有三个控制信号的方程式。每个方程只和当前状态有关（即，Q1 和 Q0 的当前值）。

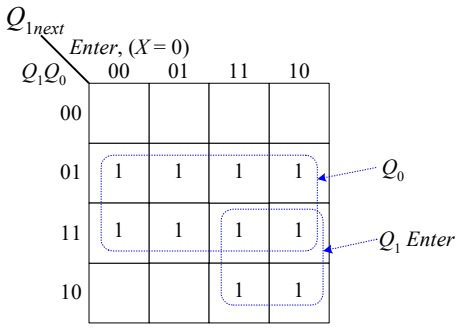
完整的控制单元电路如图 9.10（f）中所示。状态存储器由两个 D 触发器组成。到触发器的输入来自由两个下一状态方程式得出的下一状态电路。三个控制信号的输出电路从三个输出公式中分别得出。状态信号（X=0），来自数据通路中的比较器。



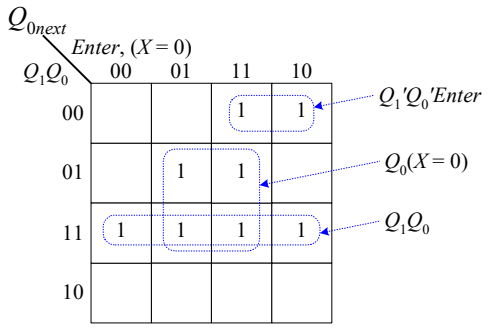
(a)

当前状态 Q_1Q_0	次态（实现） $Q_{1next}Q_{0next}$ (D_1D_0)			
	Enter, $(X = 0)$			
	00	01	10	11
00	00	00	01	01
01	10	11	10	11
10	00	00	10	10
11	11	11	11	11

(b)



$Q_{1next} = D_1 = Q_0 + Q_1Enter$



$Q_{0next} = D_0 = Q_1Q_0 + Q_0\,(X = 0) + Q_1'Q_0'Enter$

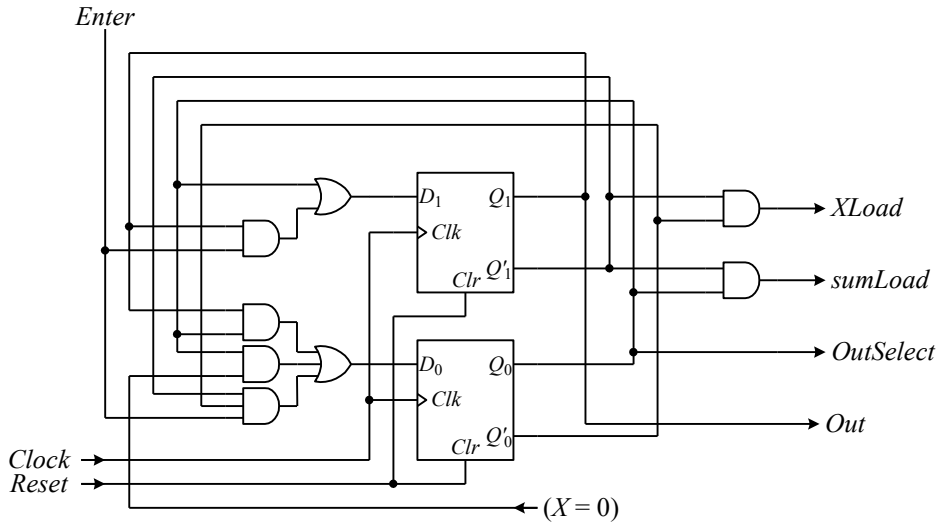
(c)

控制器子	状态 $Q_1\ Q_0$	指令	*XLoad*	*sumLoad*	*OutSelect*	*Out*
0	00	INPUT *X*	1	0	×	0
1	01	*sum* = *sum* + *X*	0	1	×	0
2	10	OUTPUT *X*	0	0	0	1
3	11	OUTPUT *sum*	0	0	1	1

(d)

$XLoad = Q_1'Q_0'$
$sumLoad = Q_1'Q_0$
$OutSelect = Q_0$
$Out = Q_1$

(e)



(f)

图 9.10 解决输入数字求和问题的控制单元：（a）状态图;（b）下一状态（实现）表;（c）K-maps 和下一状态方程;（d）控制字和输出表;（e）输出方程;（f）电路。

最终的微处理器现在可以通过连接控制单元和指定的控制以及状态信号的数据通路简单的组合形成，如图 9.11 中所示。

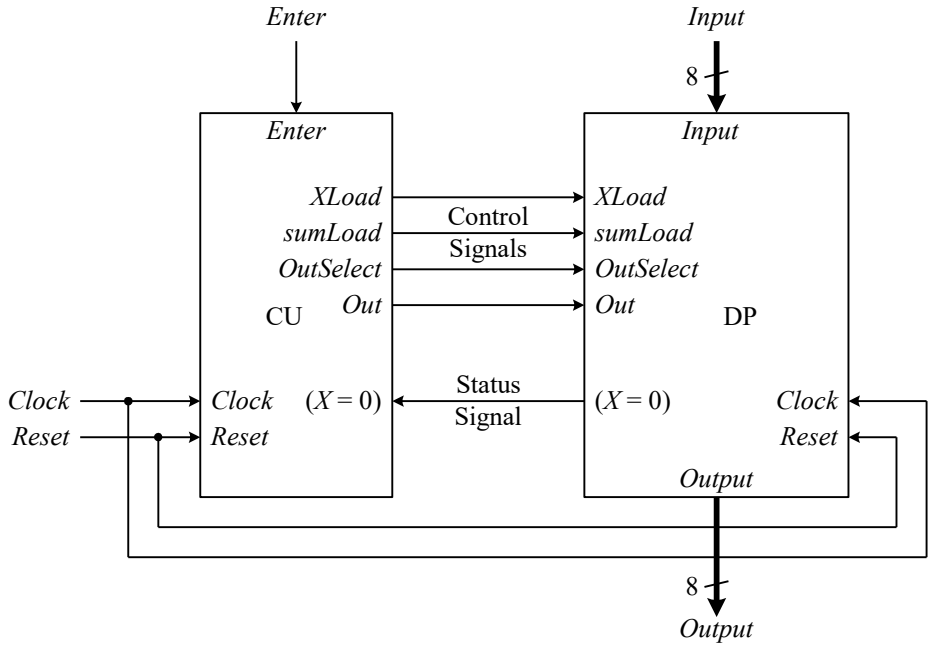


图 9.11 解决输入数字求和问题的微处理器

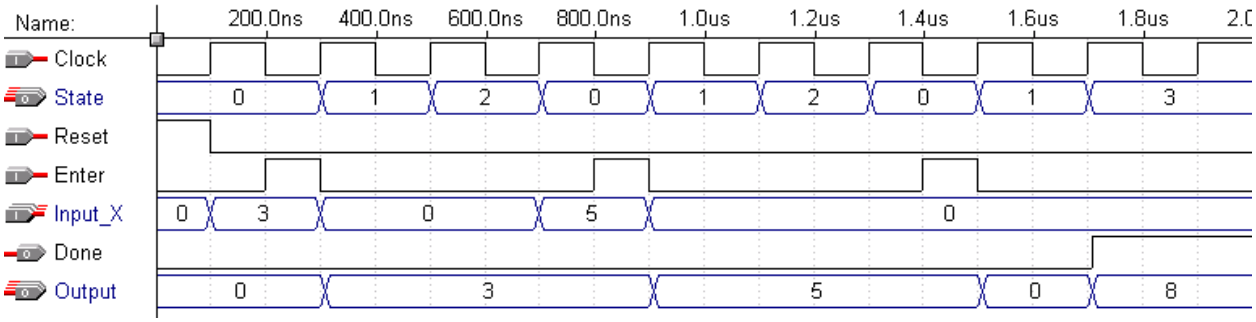


图 9.12 输入数字求和问题的简单仿真

为了在 FPGA 开发板上实现这个电路，我们需要将微处理器的 I / O 的连接到按键开关，LED 灯和时钟源上。其中，对于这个问题来说最重要的接口电路是去抖动的输入开关电路。一个简单的开关去抖动电路使用的是 D 触发器，如图 9.13 中所示。D 触发器的时钟频率必须足够慢，以保证开关有足够的时间弹跳沉降，从而使触发器锁存一个单一的值。在这里，精确的时钟频率不是特别重要。在示例中使用的时钟分频电路将 50 MHz 时钟分频为约 4Hz。

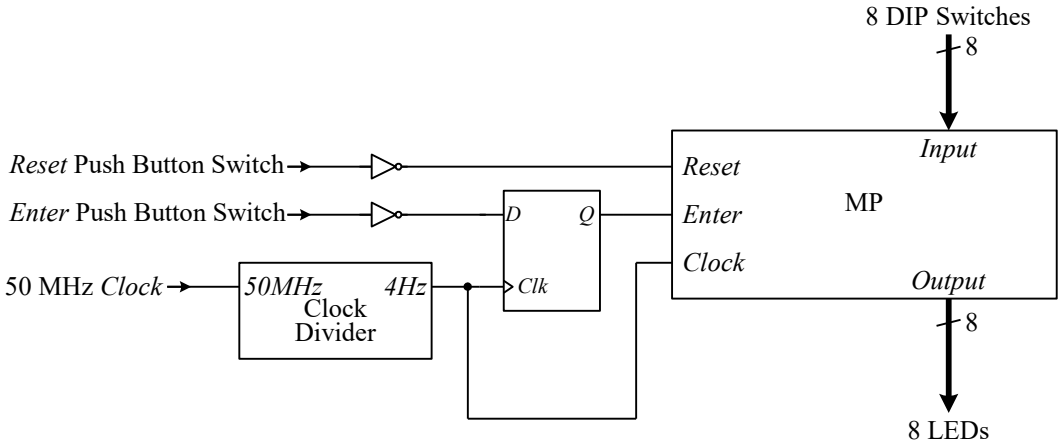
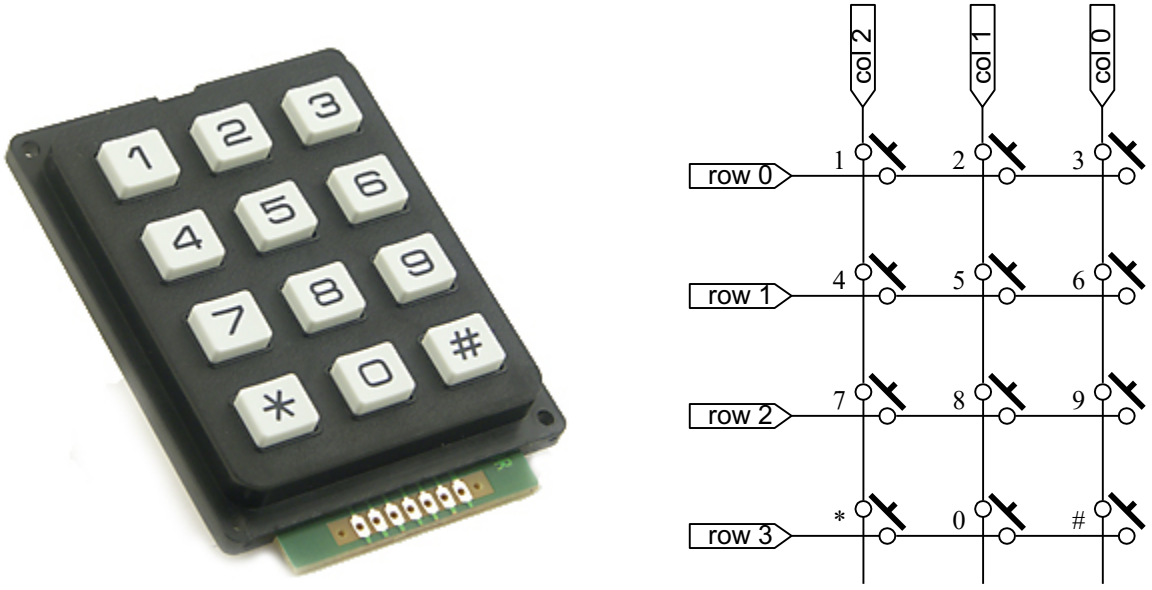


图 9.13 输入数字求和问题的微处理器外部 I/ O 接口。

9.3 3×4 键盘控制器

键盘就像按钮，并且可以有很多个这样的按钮组成一个单元。图 9.14（a）中展示了一个典型的放置在 3×4 网格上由 12 个键组成的键盘图像。通常情况下，开关按钮被用于按键。



(a) (b)

图 9.14 键盘：（a）图片；（b）原理图

9.3.1 工作原理

通常 12 键的键盘一共会有 24 个连接点，因为每个按键需要两个连接点。为了减少连接点的数量，同一行的键盘会分别连接到一根共同的行连接线上，同一列的键盘也会有一根共同的列连接线。3×4 键盘的内部连接状态如图 9.14（b）所示。运用这种连接方式，12 键键盘只需要 7 根连接线就可以了：三根列连接线和四个行连接线。

使用开关按键，每行与每列的交点通常是不连接的。当一个特定的按键被按下，该按键所在的行与列会连接起来。举例来说，如果键“1”被按下，它会连接起行 0 和列 2；当按键“8”被按下，它会连接起行 2 和列 1。

我们可以通过行与列的连接情况判断哪个键被按下。我们需要记住，不管按键是否被按下，我们还没有定义逻辑值，比如，高组态 Z。作为第一次尝试，我们可以尝试将 9.15（a）中的所有行输入连接到 VCC。我们注意到在三个列连接点，不会出现任何一个可能的逻辑值。举例来说，在第 2 列，当那一列没有键被按下的时候，下拉电阻使得该列逻辑值为 1，而当某一个该列上的按键被按下的时候，第 2 列会变成逻辑 1，因为行连接到 VCC。但是，问题是，我们是否可以分辨出该列中四个按键到底是哪个按下呢？

为了能够分辨四个按键中到底是哪个被按下的，我们不能讲所有的行都连接到 VCC。就像复用 7 段 LED 一样，我们每次只能有选择性地将某一行设置为逻辑 1 而其余行为逻辑 0。这个需要以一个非常快的速度来循环，相比于按键和松开按键的毒素。为了达到这个目的，我们需要修改电路使得它如图 9.15（b）所示。将 VCC 换成四个输入端口，每个端口可以设置为 1 或者 0。所以，当按键 0 被按下的时候，列 1 只有在行 3 是逻辑 1 的时候才会输出逻辑 1。在这个时候你可能会问这和之前的电路有什么不一样，因为对于两种电路来说，一列中的四个按钮中的任意一个被按下你在列连接线处看到的都是逻辑 1？最重要的变化是如果我们设置行 3 位逻辑 0，不管按键 0 会不会被按下列 1 都会输出逻辑 0。换句话说，因为我们知道哪一行我们会输出逻辑 1，所以当我们在列 1 得到输出逻辑 1,的时候，我们会知道哪一个节点的按键被按下。

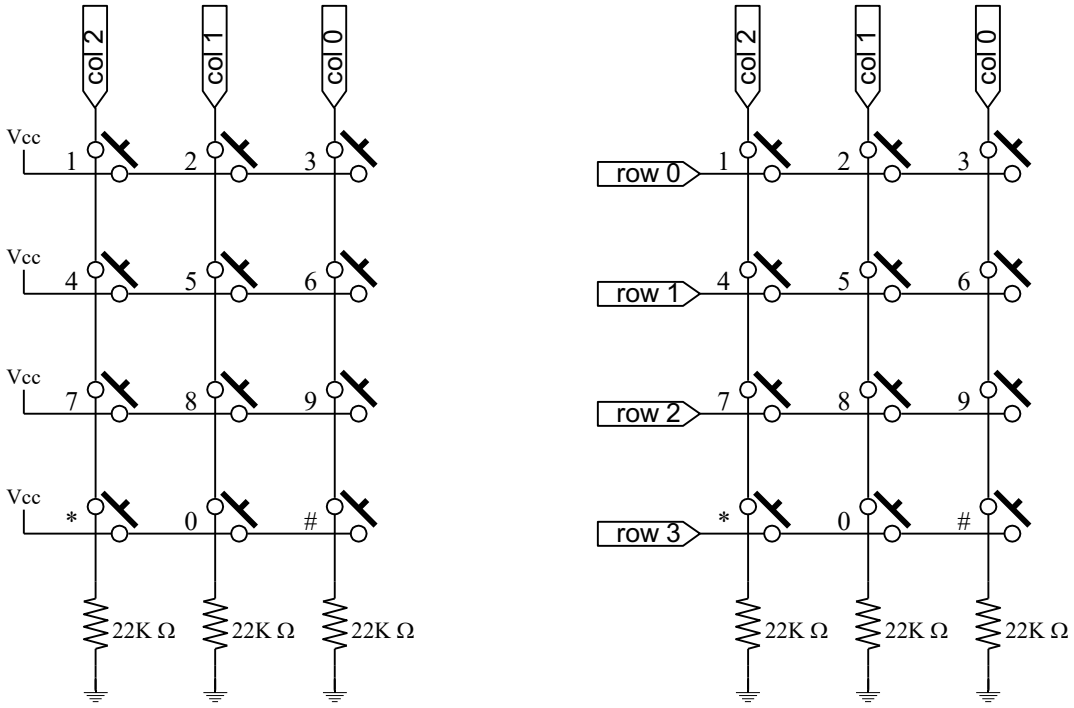


图 9.15 键盘连接：（a）不正确的连接方式；（b）正确的连接方式

为了让这个原理运作，我们需要以尽可能快的速度对每一行输入逻辑 1 或者 0，然后读取每一列到底输出的是逻辑 1 还是逻辑 0。通常会用一个控制器向某一行输出逻辑 1，其他输入逻辑 0，并且以一个非常快的速度循环这一过程。在输出 1 到某行后，然后观察三列哪一列输出逻辑 1，如果某一列有逻辑 1，说明该列和该行的交点的那一个按键被按下。

9.3.2 控制器设计

3×4 键盘控制器的状态图如图 9.16 所示。它在八个状态内循环。八个状态两两为一组。每一组内的第一个状态会输出逻辑为 1 的那一行的行号，第二个状态则检测三列的输出值。举个例子，状态 000 中，行 0 被输入逻辑 1，然后在状态 001 中，检测三列中那一列输出逻辑 1。如果列 0 输出 1 那么输出寄存器 Key 则被赋值 3（以二进制的形式）。图 9.17 显示了 3×4 键盘的 Verilog 代码。

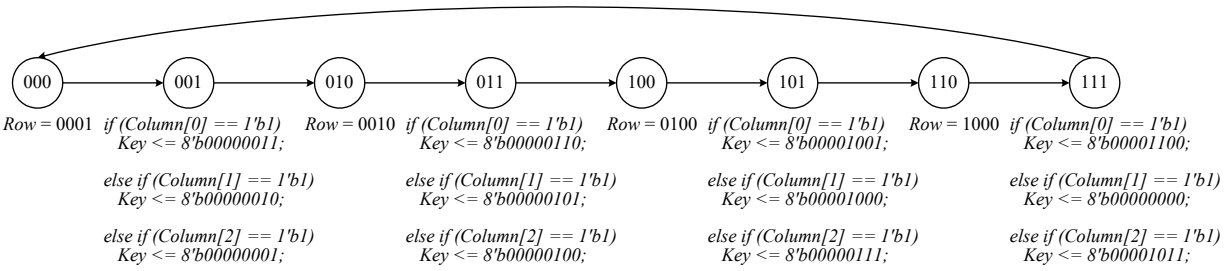


图 9.16 3×4 键盘控制器的状态图

```
module fsm (
  input Clock, Reset,
  input [2:0] Column,
  output reg [3:0] Row,
  output reg [7:0] Key
```

```
    );

  reg [2:0] state;

  always @ (posedge Clock, posedge Reset) begin
   if (Reset) begin
     state <= 3'b000;
     end
   else
     case (state)
     3'b000: begin
       // set row0 to 1
       Row <= 4'b0001;
       state <= 3'b001;
       end
     3'b001: begin
       // check key press
       if (Column[0] == 1'b1) Key <= 8'b00000011;
       else if (Column[1] == 1'b1) Key <= 8'b00000010;
       else if (Column[2] == 1'b1) Key <= 8'b00000001;
       state <= 3'b010;
       end
     3'b010: begin
       // set row1 to 1
       Row <= 4'b0010;
       state <= 3'b011;
       end
     3'b011: begin
       // check key press
       if (Column[0] == 1'b1) Key <= 8'b00000110;
       else if (Column[1] == 1'b1) Key <= 8'b00000101;
       else if (Column[2] == 1'b1) Key <= 8'b00000100;
       state <= 3'b100;
       end
     3'b100: begin
       // set row2 to 1
       Row <= 4'b0100;
       state <= 3'b101;
       end
     3'b101: begin
       // check key press
       if (Column[0] == 1'b1) Key <= 8'b00001001;
       else if (Column[1] == 1'b1) Key <= 8'b00001000;
       else if (Column[2] == 1'b1) Key <= 8'b00000111;
       state <= 3'b110;
       end
     3'b110: begin
       // set row3 to 1
       Row <= 4'b1000;
       state <= 3'b111;
       end
     3'b111: begin
       // check key press
       if (Column[0] == 1'b1) Key <= 8'b00001100;
       else if (Column[1] == 1'b1) Key <= 8'b00000000;
       else if (Column[2] == 1'b1) Key <= 8'b00001011;
       state <= 3'b000;
```

```
      end
    default: begin
      state <= 3'b000;
      end
    endcase
  end // always
endmodule
```

图 9.17 3×4 键盘的 Verilog 代码

9.4 PS2 键盘和鼠标

老式电脑的键盘和鼠标使用 PS2 协议和连接器来实现与电脑的通信。尽管这是一个老的技术，但是是一个很好的例子来解释串行命令通信。虽然二者都使用 PS2 协议，但鼠标的控制器相对来说更复杂一些，因为它需要赋予一个初始序列，而键盘则不需要。在这一小节中，我们讲解两者的构造。

9.4.1 工作原理—PS2 键盘

PS2 键盘和控制器之间的通信（在 FPGA 芯片上实现）使用两个信号：键盘时钟和键盘数据。处于未活动状态时，即键盘上没有键按下，键盘时钟和键盘数据置 1。当有键按下（或松开），键盘通过数据线向控制器连续地发送一个与按键对应的编码。键盘通过键盘时钟线发送时钟脉冲，键盘数据线上的串行数据通过时钟脉冲在键盘与控制器之间同步。

通过键盘数据线发送的对应每个按键的数据均为 11 位。这 11 位分别是：起始位 0，从最低有效位到最高有效位对应于按键编码的八个数据位，奇校验位，终止位 1。图 9.18 列出了一些按下键时键盘产生的相应的按键编码。松开键时，会产生一个不同的编码。设置奇校验位使八个数据位中 1 的个数加上校验位的总数为奇数。

图 9.19 表示了对应连字符的按键编码为 4E(二进制表示为 01001110)的数据传输采样时序图。起始为非活动状态，键盘数据线和键盘时钟线置 1，当键盘数据线的起始位设置为 0，传输开始。在每个时钟周期，键盘通过键盘数据线发送一位数据或校验位，键盘时钟线上的时钟脉冲由键盘产生。编码 4E 的校验位为 1，因为八个数据位中 1 的个数为偶数，为了实现奇校验，校验位必须设置为 1。

键	键代码
1	16
2	1E
3	26
4	25
5	2E
6	36
7	3D
8	3E
9	46
0	45

键	键代码
A	1C
B	32
C	21
D	23
E	24
F	2B
G	34
H	33
I	43
J	3B

键	键代码
K	42
L	4B
M	3A
N	31
O	44
P	4D
Q	15
R	2B
S	1B
T	2C

键	键代码
U	3C
V	2A
W	1D
X	22
Y	35
Z	1A
Esc	76
BS	66
CR	5A
Ctrl	14

图 9.18 键盘产生的按键编码的部分列表

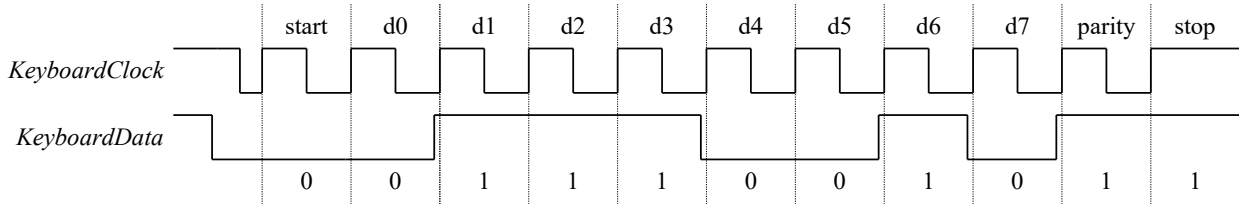


图 9.19 按键编码 4E 的数据传输采样时序图

9.4.2 控制器设计—PS2 键盘

PS2 键盘控制器的状态图 9.20(a)由图 9.19 的时序图导出。在每个数据状态 d0，d1，…，d7，我们从键盘数据输入线得到一位相应的数据。例如，假定我们使用一个 8 位寄存器（命名为 Keycode）存储八个数据位。在状态 d0，我们将键盘数据的值赋给$Keycode_0$（也就是 Keycode 的第 0 位），在状态 d1，我们将键盘数据的值赋给$Keycode_1$，对所有的数据位进行同样的处理。这是考虑到在有限状态机中从一个状态到另一个状态的转换与时钟信号键盘时钟同步。简单来说，我们不需要检查起始位、校验位或者终止位。

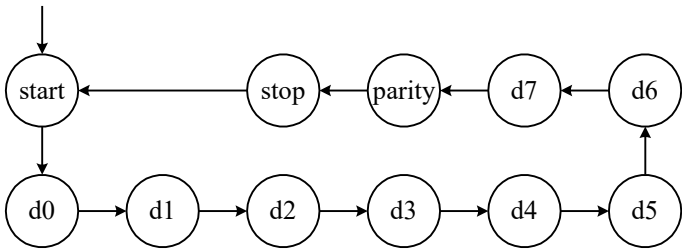
我们注意到状态图中没有等待状态等待起始位，这看起来是错误的，好像在没有按下键时有限状态机也会遍历所有的状态。然而，事实并非如此，因为驱动有限状态机的时钟并非连续不断地运转。我们用键盘时钟信号驱动有限状态机的时钟（看图 9.20(f)中的时钟模块连接），因为非活动状态时信号始终为 1，所以 FSM 会停留在起始状态等待键按下。一旦有键按下，键盘时钟开始切换，从而激活有限状态机，并定时记录数据位。

图 9.20(b)中展示了使用了四个 D 触发器对的十一个状态进行编码的次态表。从次态表导出的卡诺图和次态方程如图 9.20(c)所示。

实际上，控制器电路不能控制键盘的工作，因为它无法产生对键盘的控制信号。相反地，它接收从键盘发送来的串行数据信号，并将其封装成数据字节。控制器的输出是代表着按键编码的数据字节。输出表如图 9.20(d)所示，相应的输出方程如图 9.20(e)所示。在状态 d0，键盘数据线上的位加载 Keycode 寄存器的第 0 位；在状态 d1，键盘数据线上的位加载 Keycode 寄存器的第 1 位；依次类推。因此，Keycode 寄存器的每一位都必须能够独立加载键盘数据，每根加载使能线都由相应的状态编码决定。因此，图 9.20(e)中每个输出方程都不是简单地用 5 输入与门实现的。相反地，每个输出信号都需要用一个带使能端的 D 触发器。触发器的 D 输入端与键盘数据线相连，加载使能端 E 由相应状态的四个 Q 值的与决定。

通过用次态方程表示下一状态电路，用 D 触发器实现状态存储，用输出方程表示输出电路，我们获得了完整的控制器电路，如图 9.20(f)所示。控制器电路的连接和实现如图 9.20(g)所示。

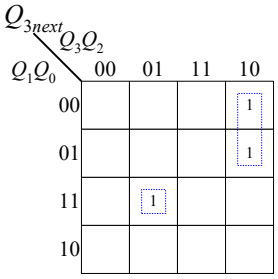
该控制器的设计并不是接收 PS2 键盘发送的数据的唯一方法。实际上，一个简单的移位寄存器就可以实现该功能。



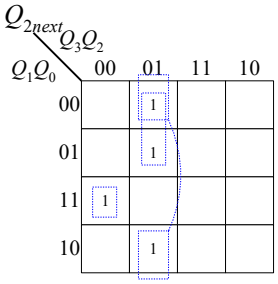
(a)

当前状态 $Q_3Q_2Q_1Q_0$		次态(实现) $Q_{3next}Q_{2next}Q_{1next}Q_{0next}$ ($D_3D_2D_1D_0$)
0000	start	0001
0001	d0	0010
0010	d1	0011
0011	d2	0100
0100	d3	0101
0101	d4	0110
0110	d5	0111
0111	d6	1000
1000	d7	1001
1001	parity	1010
1010	stop	0000

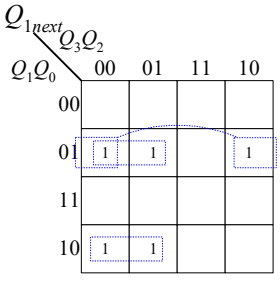
(b)



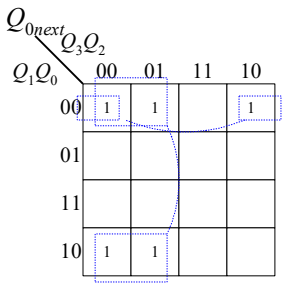
$$Q_{3next} = D_3 = Q_3Q_2'Q_1' + Q_3'Q_2Q_1Q_0$$



$$Q_{2next} = D_2 = Q_3'Q_2Q_1' + Q_3'Q_2Q_0' + Q_3'Q_2'Q_1Q_0$$



$$Q_{1next} = D_1 = Q_3'Q_1'Q_0 + Q_2'Q_1'Q_0 + Q_3'Q_1Q_0'$$



$$Q_{0next} = D_0 = Q_3'Q_0' + Q_2'Q_1'Q_0'$$

(c)

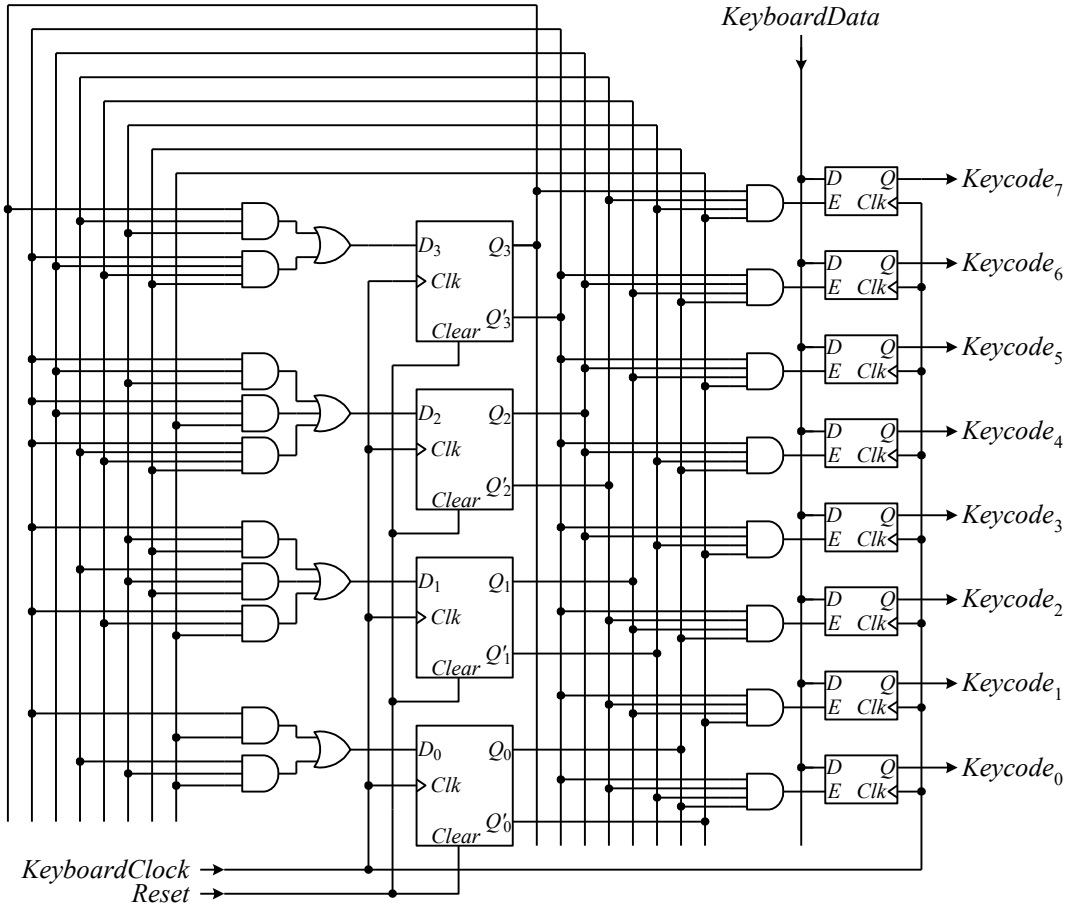
当前状态 $Q_3Q_2Q_1Q_0$		输出 $Keycode_{7\text{-}0}$
0000	start	
0001	d0	– – – – – – – *KeyboardData*
0010	d1	– – – – – – *KeyboardData* –
0011	d2	– – – – – *KeyboardData* – –
0100	d3	– – – – *KeyboardData* – – –
0101	d4	– – – *KeyboardData* – – – –
0110	d5	– – *KeyboardData* – – – – –
0111	d6	– *KeyboardData* – – – – – –
1000	d7	*KeyboardData* – – – – – – –
1001	parity	
1010	stop	

Where a dash (–) means no change to that *Keycode* bit.

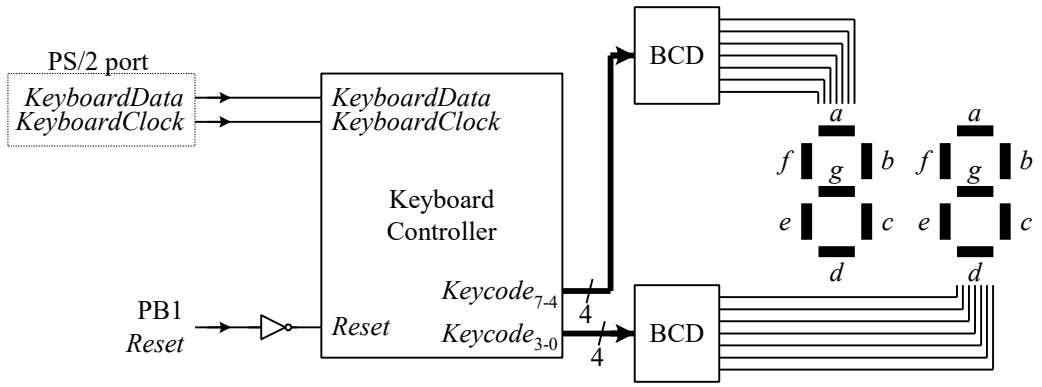
(d)

$$Keycode_7 = Q_3Q_2'Q_1'Q_0'\ KeyboardData$$
$$Keycode_6 = Q_3'Q_2Q_1Q_0\ KeyboardData$$
$$Keycode_5 = Q_3'Q_2Q_1Q_0'\ KeyboardData$$
$$Keycode_4 = Q_3'Q_2Q_1'Q_0\ KeyboardData$$
$$Keycode_3 = Q_3'Q_2Q_1'Q_0'\ KeyboardData$$
$$Keycode_2 = Q_3'Q_2'Q_1Q_0\ KeyboardData$$
$$Keycode_1 = Q_3'Q_2'Q_1Q_0'\ KeyboardData$$
$$Keycode_0 = Q_3'Q_2'Q_1'Q_0\ KeyboardData$$

(e)



(f)



(g)

图 9.20 PS2 键盘的控制器：(a)状态图；(b)次态图；(c)卡诺图和激励方程；(d)输出表；(e)输出方程；(f)控制器电路；(g)接口电路。

破折号(—)表示那一位 Keycode 没有变化

9.4.3 工作原理—PS2 键盘

PS2 鼠标使用与 PS2 键盘使用相同的 PS2 连接和协议。然而，PS2 鼠标控制器的设计比键盘要有趣一些，因为鼠标需要获得控制器发来的初始化命令，并且数据线和时钟线上的信号都是双向的。

上电后，按下一个键，键盘会立即发送数据字节到控制器。另一方面，在鼠标发送数据字节到控制器之前，需要一个初始化命令来告诉它要做什么。初始化之后，鼠标会连续地通过鼠标数据线发送数据到控制器，同时像键盘一样受到鼠标时钟线上的时钟控制。在这里，我们用不同的名字来命名数据和时钟线，但实际上它们是 PS2 连接器上相同的信号线。控制器和鼠标利用同样的鼠标数据线和鼠标时钟线给彼此发送信息，因此这两根线必须是双向的，这也增加了控制器设计的复杂度。此外，由于控制器需要控制鼠标时钟线，所以有限状态机不能通过鼠标时钟线控制时钟，因此，我们需要一个独立的时钟。

当控制器要给鼠标发送数据，比如说上电时的初始化信息，控制器必须首先把鼠标时钟和鼠标数据拉低至少 100 微秒从而抑制通信，使鼠标时钟线和鼠标数据线置于“请求—发送”状态，然后将鼠标时钟设置为高阻抗释放。释放鼠标时钟线后，控制器必须等待鼠标获得时钟线的控制权。此时，鼠标会在鼠标时钟线上产生时钟信号，控制器将在鼠标数据线上发出初始化指令 F4，从最低有效位开始每个时钟脉冲发送一位。和键盘类似，发送的每个字节都要加上奇校验位和终止位。对于命令 F4，奇校验位是 0，终止位始终是 1。

发送终止位后，鼠标控制器会将鼠标数据线设为高阻抗并将其释放。鼠标将会获得鼠标数据线的控制权并发送应答字节 FA 到控制器。接着，应答字节发送最低有效位、奇校验位和终止位。初始化序列到此结束。

鼠标被初始化之后，将通过鼠标时钟线的时钟控制在鼠标数据线上连续地发送数据到控制器。数据位被分为三个数据包的帧，并且每个数据包包含 11 位（起始位 0，8 位数据位，奇校验位和终止位 1）。每个帧中的三个字节的数据比特的解释如下：

Bit	7	6	5	4	3	2	1	0
Byte 1	Yo	Xo	Ys	Xs	1	M	R	L
Byte 2	X_7	X_6	X_5	X_4	X_3	X_2	X_1	X_0
Byte 3	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0

M，R 和 L 分别是鼠标中键、右键和左键的状态（1 = 按下；0 = 松开），
Y_0和X_0分别表示 Y 方向和 X 方向上的溢出位，
Y_s表示 Y 方向上的符号位（1 = 向下移动；0 = 向上移动），
X_s表示 X 方向上的符号位（1 = 向左移动；0 = 向右移动），
X_7—X_0表示 X 方向水平移动距离的二进制补码（负 = 向左移动；正 = 向右移动），
Y_7—Y_0表示 Y 方向垂直移动距离的二进制补码（负 = 向下移动；正 = 向上移动）。

下表显示了每个鼠标操作对应接收到的三个字节

操作	字节 3	字节 2	字节 1
鼠标中键点击	00	00	0C
鼠标右键点击	00	00	0A
鼠标左键点击 ick	00	00	09
左移	00	FF	18
右移	00	01	08
下移	FF	00	28
上移	01	00	08

如上所述，总结步骤。在图 9.22 的代码解释中有对这些步骤的注解。
1、拉低时钟和数据线至少 100 微秒。
2、释放时钟线。

3、等待时钟线释放（高）。
4、从最低有效位开始，发送初始化命令字节 F4（16 进制）。
5、等待设备来拉低时钟。
6、等待设备来拉高时钟。
7、对其余的 7 个数据位、奇校验位和终止位重复步骤 5-7。
8、释放数据线。
9、接收应答字节 FA。
10、接收三个数据包的数据，每个数据包含 11 位。
11、重复步骤 10 和 11。

9.4.4 控制器设计—PS2 键盘

控制 PS2 鼠标的专用微控制器不需要数据通道。用行为级 Verilog 和 VHDL 代码编写的有限状态机如图 9.21 和 9.22 所示。请注意，鼠标时钟和鼠标数据信号为双向信号。鼠标控制器的有限状态机状态转换步骤如上所述。对于所有的状态，如果想检查鼠标时钟信号，并不是直接对它进行检查，而是检查与原始鼠标时钟信号相比有微小时间延迟的滤过鼠标时钟信号，代码中第二个进程在下一个时钟上升沿分配鼠标时钟信号到滤过鼠标时钟信号，从而增加了轻微的时钟延迟。

```
module MouseController (
  input Clock, Reset,
  inout reg MouseClock, MouseData,
  output  LeftButton, RightButton, MidButton,
  output [7:0] Byte1, Byte2, Byte3,
  output reg [7:0] Debug
);

  reg [43:0] packet;
  reg [5:0] count;
  reg [9:0] command;
  reg FilteredMouseClock;

  reg [3:0]state;
  parameter s_init1=0, s_init2=1, s_init3=2,
       s_cmd1=3, s_cmd2=4,
       s_cmd20=5, s_cmd21=6,
       s_ack1=7, s_ack2=8,
       s_recv1=9, s_recv2=10,
       s_end=11;

  assign LeftButton = packet[1];
  assign RightButton = packet[2];
  assign MidButton = packet[3];
  assign Byte1 = packet[8:1];
  assign Byte2 = packet[19:12];
  assign Byte3 = packet[30:23];

  // Next-state logic
  always @ (posedge Clock or posedge Reset) begin
    if (Reset) begin
      MouseClock <= 1'b0;  // 1)
      MouseData <= 1'b0;
      packet <= {44{1'b0}}; // 44 0's
      command <= 10'b1011110100;     // stop, odd parity, F4
      count <= 0;
```

```
    state <= s_init1;
    Debug <= 8'b00000000;
  end else
    case (state)
    s_init1: begin // 2)
      MouseClock <= 1'b0;
      MouseData <= 1'b0;
      state <= s_init2;
      end
    s_init2: begin // 3)
      MouseClock <= 1'bz;
      MouseData <= 1'b0;
      state <= s_init3;
      end
    s_init3: begin // 4) wait for clock to go hi
      if (FilteredMouseClock == 1'b1) begin
        count <= 0;
        state <= s_cmd1;
        end
      end
    s_cmd1: begin // 5) send out command F4, LSB first
      MouseData <= command[count];
      if (FilteredMouseClock == 1'b0) begin    // 6)
        if (count < 10) begin
          count <= count + 1;
          state <= s_cmd2;
          end
        else
          state <= s_cmd21;
        end
      end
    s_cmd2: begin // 7)
      if (FilteredMouseClock == 1'b1) begin
        state <= s_cmd1;
        end
      end
    s_cmd21: begin // 9)
      Debug <= 8'b00000001;
      MouseData <= 1'bZ;  // release the data line
      if (FilteredMouseClock == 1'b1) begin
        count <= 0;
        state <= s_ack1;
        end
      end
    // at this point, if you have a mouse with an IR instead of a
    // mechanical ball, the LED on the bottom should be on

    // get acknowledge message FA
    s_ack1: begin
      Debug <= 8'b00000010;
      packet[count+22] <= MouseData;
      if (FilteredMouseClock == 1'b0) begin
        if (count < 11) begin
          count <= count + 1;
          state <= s_ack2;
          end
        else begin
```

```
          count <= 0;
          state <= s_recv1;
          end
        end
      end
     s_ack2: begin
      Debug <= 8'b00000011;
      if (FilteredMouseClock == 1'b1) begin
        state <= s_ack1;
        end
      end
     // Receive data from mouse
     s_recv1: begin
      Debug <= 8'b00000100;
      packet[count] <= MouseData;
      if (FilteredMouseClock == 1'b0) begin
        if (count < 33) begin
          count <= count + 1;
          state <= s_recv2;
          end
        else begin
          count <= 0;
          state <= s_recv1;
          end
        end
      end
     s_recv2: begin
      Debug <= 8'b00000101;
      if (FilteredMouseClock == 1'b1) begin
        state <= s_recv1;
        end
      end
     default: begin
      Debug <= 8'b11111111;
      end
     endcase
  end  // always

// slight delay for the mouse clock
  always @ (posedge Clock) begin
    FilteredMouseClock <= MouseClock;
    end

endmodule
```

图 9.21 PS2 鼠标控制器的行为级 Verilog 代码

```
LIBRARY IEEE;
USE  IEEE.STD_LOGIC_1164.ALL;

ENTITY MouseController IS PORT (
  Reset: IN STD_LOGIC;
  Clock: IN STD_LOGIC;
  MouseClock: INOUT STD_LOGIC;
  MouseData: INOUT STD_LOGIC;
  LeftButton,RightButton,MidButton: OUT STD_LOGIC;
  Byte1: OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
```

```
  Byte2: OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
  Byte3: OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
  Debug: OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
  );
END MouseController;

ARCHITECTURE Behavioral OF MouseController IS
  TYPE state_type IS (
     s_init1,s_init2,s_init3,
     s_cmd1,s_cmd2,
     s_cmd20,s_cmd21,
     s_ack1,s_ack2,
     s_recv1,s_recv2,
     s_end);
  SIGNAL state: state_type;

  SIGNAL packet: STD_LOGIC_VECTOR(43 DOWNTO 0);
  SIGNAL count: INTEGER RANGE 0 TO 64;
  SIGNAL command: STD_LOGIC_VECTOR(9 DOWNTO 0);
  SIGNAL FilteredMouseClock: STD_LOGIC;

BEGIN

  LeftButton <= packet(1);
  RightButton <= packet(2);
  MidButton <= packet(3);
  Byte1 <= packet(8 DOWNTO 1);
  Byte2 <= packet(19 DOWNTO 12);
  Byte3 <= packet(30 DOWNTO 23);

  FSM: PROCESS(Clock, Reset)
  BEGIN
    IF (Reset = '1') THEN
      MouseClock <= '0';   -- 1)
      MouseData <= '0';
      packet <= (OTHERS => '0');
      command <= "1011110100";       -- stop, odd parity, F4
      count <= 0;
      state <= s_init1;
      Debug <= "00000000";
    -- this FSM is driven by the DE2 clock signal
    ELSIF (Clock'EVENT AND Clock = '1') THEN
      CASE state is
      WHEN s_init1 =>   -- 2)
        MouseClock <= '0';
        MouseData <= '0';
        state <= s_init2;
      WHEN s_init2 =>   -- 3)
        MouseClock <= 'Z';
        MouseData <= '0';
        state <= s_init3;
      WHEN s_init3 =>   -- 4) wait for clock to go hi
        IF (FilteredMouseClock = '1') THEN
          count <= 0;
          state <= s_cmd1;
        END IF;
      WHEN s_cmd1 => -- 5) send out command F4, LSB first
```

```
      MouseData <= command(count);
      IF (FilteredMouseClock = '0') THEN -- 6)
        IF (count < 10) THEN
          count <= count + 1;
          state <= s_cmd2;
        ELSE
          state <= s_cmd21;
        END IF;
      END IF;
    WHEN s_cmd2 => -- 7)
      IF (FilteredMouseClock = '1') THEN
        state <= s_cmd1;
      END IF;

    WHEN s_cmd21 =>   -- 9)
      Debug <= "00000001";
      MouseData <= 'Z';  -- release the data line
      IF (FilteredMouseClock = '1') THEN
        count <= 0;
        state <= s_ack1;
      END IF;
    -- at this point, if you have a mouse with an IR instead of a
                                  mechanical ball, the LED on the bottom
                                  should be on

    -- get acknowledge message FA
    WHEN s_ack1 =>
      Debug <= "00000010";
      packet(count+22) <= MouseData;
      IF (FilteredMouseClock = '0') THEN
        IF (count < 11) THEN
          count <= count + 1;
          state <= s_ack2;
        ELSE
          count <= 0;
          state <= s_recv1;
        END IF;
      END IF;
    WHEN s_ack2 =>
      Debug <= "00000011";
      IF (FilteredMouseClock = '1') THEN
        state <= s_ack1;
      END IF;
    -- Receive data from mouse
    WHEN s_recv1 =>
      Debug <= "00000100";
      packet(count) <= MouseData;
      IF (FilteredMouseClock = '0') THEN
        IF (count < 33) THEN
          count <= count + 1;
          state <= s_recv2;
        ELSE
          count <= 0;
          state <= s_recv1;
        END IF;
      END IF;
    WHEN s_recv2 =>
```

```
        Debug <= "00000101";
        IF (FilteredMouseClock = '1') THEN
          state <= s_recv1;
        END IF;

      WHEN OTHERS =>
        Debug <= "11111111";
      END CASE;
    END IF;
  END PROCESS;

-- slight delay for the mouse clock
  PROCESS
  BEGIN
    WAIT UNTIL Clock'event and Clock = '1';
    FilteredMouseClock <= MouseClock;
  END PROCESS;

END Behavioral;
```

图 9.22 PS2 鼠标控制器的行为级 VHDL 代码

9.5 RS-232 蓝牙通信控制器

蓝牙连接通常用于短程内两台设备之间的无线通信。例如, 经常使用手机蓝牙连接无线扬声器和耳机。我们将使用 HC-06 从蓝牙模块,这是一种廉价和容易买到的蓝牙模块，用于实现一个我们将要设计的专用微处理器控制器和装在一台计算机或者一个手机上的一个主蓝牙主机之间的无线通信。图 9.23 显示了一幅 HC-06 蓝牙从模块。该蓝牙模块工作原理就像一个串行调制解调器,为了使用这个蓝牙模块，我们需要理解 RS-232 串行协议是如何工作的。

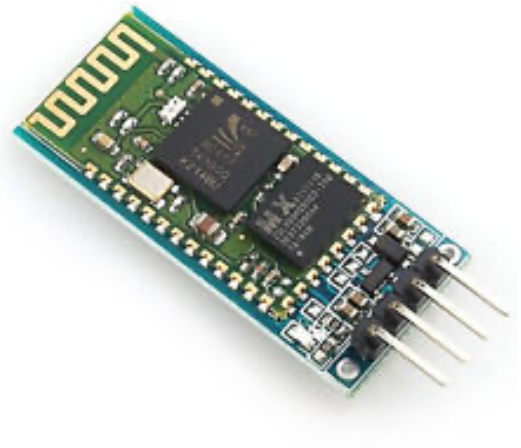


图 9.23 HC-06 从蓝牙模块

RS-232 串口曾经是用于个人电脑连接外围设备的一个标准特征，就像调制解调器, 鼠标电脑和打印机一样。在现代个人电脑上，USB 连接完全取代了 RS-232。尽管如此，由于 RS-232 有着相比 USB 更简单的协议，许多 RS-232 设备仍在使用，尤其是在工业机器，科学仪器，通讯设备，以及我们的例子中的蓝牙模块。

9.5.1 RS-232 工作原理

RS-232 协议定义了两个设备之间的通信。主要设备，如计算机，被称为 DTE（数据终端设备），次要的外围设备，比如科学仪器，被称为 DCE（数据电路终接设备或最初定义为数据通信设备）。RS-232 的物理连接器具有 9 个（DB-9）或 25 个（DB-25）引脚，但通常只有 3 个针（引脚 2、3 和 5）被用于两个通信设备之间的连接。引脚 2 和 3 用于数据发送和接收信号, 而引脚 5 接地。引脚 2 是传送还是接收信号取决于是否用在 DTE 端或 DCE 端。在 DTE 端，引脚 2 是接收（RxD）信号，引脚 3 是传输（TxD）信号。在 DCE 端，刚好相反。一个典型的计算机与外围设备或控制器之间的连接是把引脚 2 连到一端，引脚 3 连到一端另一端。反之亦然。DTE 端的引脚 5 和 DCE 端的引脚 5 连接起来。

不同于 PS2 的连接，RS-232 没有用于同步两个设备的时钟线。相反，一个固定的时钟速率（称为波特率）必须预先被确定，并且经过两个设备的确认。典型的波特率是 9600、38400 和 115200。波特率通常被认为是每秒钟比特的数量，但也并非总是如此。此外，校验位和停止位也必须预先确定并且经过两个设备的确认。

校验位用于错误检查。RS-232 协议允许偶校验，奇校验或者没有校验。对于偶数和奇数校验，校验位的设置或重置取决于 8 位数据中 1 的个数。具体，对于偶校验，8 位数据中 1 的个数加上校验位必须是偶数。而对于奇校验，1 的个数必须是奇数。例如，如果 8 位数据位是 01001011（其中有 4 个 1），于是，对于偶校验，校验位必须被复位为 0（使 1 的个数为偶数），但对于奇校验，校验位要设为 1（使 1 的个数为奇数）。通过检查校验位，接收器可以在一些数据在数据传输中被翻转时，有所察觉。细心的读者可能会注意到有些错误并没有被发现。没有校验仅仅意味着在传输过程中，不使用校验位。

RS-232 数据传输总是以一个起始位开始，然后紧接着有八个数据位（最低有效位在前）。当最后一个数据位被（这是最重要的一位）发送出去，还可以再加上一个校验位，以及一个或多个停止位。是否有一个奇偶校验位或多少停止位是预先约定的，并且被设置在通信的两个设备中。RS-232 的通常配置经常用缩略词“N81”来表示，它是指“没有校验位，8 位数据位，1 停止位”。在这样的配置下，发送一个字节的数据用了 10 个位（一个起始位，8 个数据位，1 个停止位）。在 N81 配置下，一个 8 位数据 01001011 的波形如图 9.24 所示。

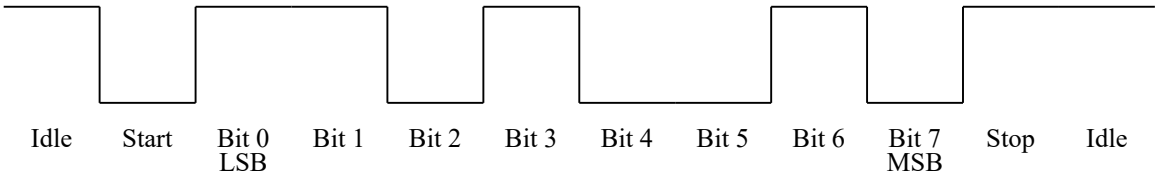


图 9.24 八位数据在 N81 配置下的 RS-232 波形

9.5.2 RS-232 控制器设计

RS-232 发送方控制器的 Verilog 代码如图 9.25 所示。发送方基本上是一个并行到串行的转换器。当有数据要发送时，8 位数据被加载到数据寄存器中，并且添加起始信号。FSM（有限状态机）看到起始信号，就进入 s_start 状态，将起始信号发送到传输 TxD 行。然后进入 s_send 状态，发送寄存器 data 中的 8 位数据，每位数据一个时钟周期，从最低有效位开始。为了正常工作，FSM 的时钟速度必须运行在正确的波特率。

```
// RS232 Sender
// Using N81
// The input clock must be set to the correct baud rate
module RS232Send (
  input clk,
  input reset,
  input [7:0] data, // 8-bit input data
  input start,      // start transmission
  output reg TxD,   // serial transmit output signal
  output reg done
```

```
);

  parameter s_idle=0, s_start=1, s_send=2, s_stop=3;
  reg [1:0] state;
  reg [3:0] count;

  always @(posedge clk, posedge reset) begin
    if (reset) begin
      TxD = 1;
      done = 0;
      state = s_idle;
    end else
      case (state)
      s_idle: begin  // idle
        TxD = 1;
        done = 0;
        count = 0;
        if (start) state = s_start;
        end
      s_start: begin // Start bit
        TxD = 0;
        done = 0;
        count = 0;
        state = s_send;
        end
      s_send: begin  // 8 data bits (least significant bit first)
        TxD = data[count];
        done = 0;
        // blocking assignment, so ordering matters
        count = count + 1;
        if (count > 7) state = s_stop;  // test is AFTER the count increment
        end
      default: begin // Stop bit
        TxD = 1;
        done = 1;
        state = s_idle;
        end
    endcase
  end
endmodule
```

图 9.25 RS-232 发送方控制器的 Verilog 代码

RS-232 接收控制器的 VHDL 代码如图 9.26 所示。不像发送方，接收方就是一个简单的串行到并行的转换器。最初，FSM 处在 s_init 状态，等待 RxD 行上的起始位。起始位是一个从 1 到 0 的转变。在在收到起始位后，FSM 进入数据接收状态 s_receive，接受 8 位数据。传入的数据是串行比特流，按照预先设定的波特率。如果 FSM 时钟设置在正确的波特率，那么所有的 FSM 需要做的就是简单地按照每个时钟周期一位数据的方式，把传入的数据送到输出寄存器 data 中。在实现中，需要特别注意的是，由于发送方在时钟的上升沿发送比特，如果接收方也是在时钟的上升沿接收比特，那么，接收者可能会在时钟上升沿之前，接受到之前的比特值，因为发送方的时钟可能会与接收方的时钟稍微偏离。为了解决这个问题，最好是在时钟周期的中间接受数据，而不是在时钟周期的开始。一个简单的实现方法是在时钟的下降沿接收数据，这刚好发生在时钟周期的中间。在收到最后一个数据位和停止位后，发出一个完成信号 done，来通知控制器它已经收到了一个字节的数据。

```
-- RS232 Receiver
-- Using N81
-- The input clock must be set to the correct baud rate
```

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

Entity RS232Receive IS PORT (
  clk: IN STD_LOGIC;
  reset: IN STD_LOGIC;
  RxD: IN STD_LOGIC;   -- serial receive input signal
  data: OUT STD_LOGIC_VECTOR(7 DOWNTO 0);      -- 8-bit output data
  done: OUT STD_LOGIC);
END RS232Receive;

ARCHITECTURE Behavioral OF RS232Receive IS
  TYPE state_type IS (s_idle, s_receive, s_stop);
  SIGNAL state: state_type;
  SIGNAL count: INTEGER RANGE 0 TO 7;
BEGIN
  PROCESS(clk, reset)
  BEGIN
    IF (reset = '1') THEN
      data <= "00000000";
      done <= '0';
      state <= s_idle;
    -- receive bits at the falling clock edge
    ELSIF (clk'EVENT AND clk = '0') THEN
      CASE state IS
        WHEN s_idle =>
          IF (RxD = '0') THEN  -- wait for start bit
            data <= "00000000";
            done <= '0';
            count <= 0;
            state <= s_receive;
          END IF;
        WHEN s_receive =>
          data(count) <= RxD;
          done <= '0';
          -- non-blocking assignment, so ordering doesn't matter
          count <= count + 1;
          IF (count > 6) THEN  -- test is BEFORE the count increment
            state <= s_stop;
          END IF;
        WHEN OTHERS =>   -- Stop bit
          done <= '1';
          state <= s_idle;
      END CASE;
    END IF;
  END PROCESS;
END Behavioral;
```

图 9.26 RS-232 接收方控制器的 VHDL 代码

9.5.3 应用实现

HC-06 蓝牙从模块可以搭配任何蓝牙主装置，比如个人电脑或手机。蓝牙主装置和从装置之间的最大距离大约是 50 英尺。蓝牙通信的细节操作封装在模块中，所以对用户来说，不需要理解蓝牙通信具体是如何工作的。用户只需要知道如何使用 RS-232 协议来使用模块接口，因为模块运作起来就像一个 RS-232 DCE 设备。作为蓝牙主装置的个人电脑，HC-06 蓝牙从模块和 RS-232 控制器之间的连接设置如图 9.27 所示。

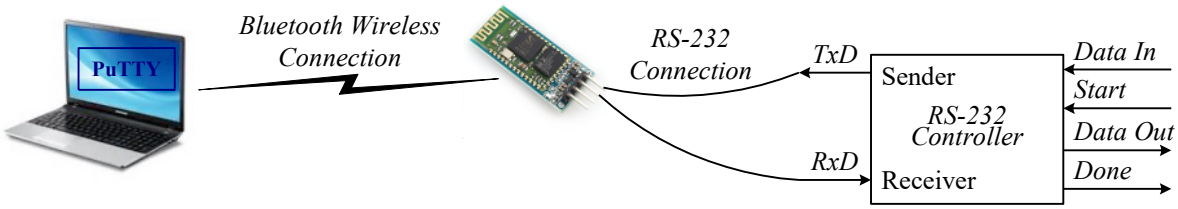


图 9.26 作为蓝牙主装置的个人电脑，HC-06 蓝牙从模块和 RS-232 控制器之间的连接设置

HC-06 蓝牙模块的默认设置如下：

波特率	9600
协议	N81（没有校验位，8 位数据，1 位停止位）
配对名	linvor
密码	1234

该模块可以通过设置控制面板中的设备和打印机，很容易地匹配到一个 Windows 个人电脑。选择添加设备。等一会儿，你的蓝牙设备（使用默认名称 ”linvor” ）将会出现。选择你的设备并继续。当需要密码时，输入默认密码 1234。你的蓝牙设备就会显示在列表中。

在个人电脑上，可以使用终端模拟器，比如 PuTTY，与蓝牙通信模块。在 PuTTY 配置界面，选择串行连接类型并输入正确的 COM 端口和传输速度。PuTTY 将字符发送到 RS-232 接收控制器的 8 位数据输出线上，同时，RS-232 发送方控制器将数据发到 PuTTY 中，这些数据将显示在 PuTTY 的终端屏幕上。

9.6 液晶显示控制器

16×2 的液晶显示器（LCD）如图 9.28 所示，是一个很常见的显示模块，它可以很容易地连接到微控制器。在这一节中，我们将介绍它的基本操作，并设计一个专门的微处理器来控制和显示一些示例字符。本节的讨论不包括模块的全部操作。该液晶显示模块完整和详细的描述和操作可以查阅制造商提供的数据手册。



图 9.2 一块 16×2 液晶显示器

9.6.1 操作原理

16×2 的 LCD 可以显示 2 行,每行 16 个字符，每个字符由一个 5×7 点阵格式构成。显示器内部有一个集成电路控制其操作，如如何控制每个字符矩阵的每个点打开和关闭以形成各种不同的字符，在哪里显示字符，是否显示光标，是否移位字符等功能。因此，将 LCD 连接到微处理器的本质是理解与内部集成电路正确联系所需的命令。除了电源线和地线，LCD 有三个控制信号（RS, R/W 和使能信号），还有八位或四位数据线。如果只用四位数据线，则只有高四位（D7～D4）连接，每个字节将被作为两个单独的十六进制数一个接一个地发出，其中最重要的十六进制数字先发送。RS 信号用来确定随后的数据位被解释为命令还是数据。如果 RS= 0，则这一位是一个命令，如果 RS= 1，则这一位是数据。R/W 信号确定数据读或写，0 = 写，1 =读。在使能信号的下降沿，RS, R/W 和数据信号被锁存到内部寄存器。

LCD 和专用微处理器之间的物理连接如图 9.29 所示。正如图中所看到的那样，三个控制信号和八个数据线的连接是非常直观的。如果选择使用 4 位数据线，则低四位（D3-D0）不用连接。现在的问题就是设计一个专用微处理器，来正确地联系 LCD，并发送必要的命令给它。

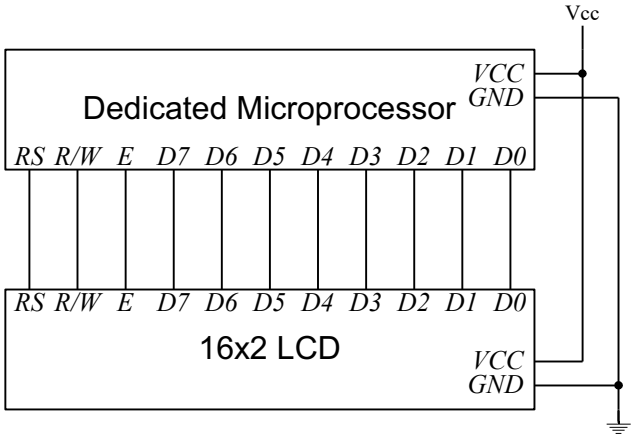


图 9.29 LCD 和专用微处理器之间的连接

在上电时，LCD 需要以一系列的命令来初始化，如图 9.30 所示。这个初始化过程包括一个十六进制代码的序列，用来告知 LCD：4 位或 8 位数据是否将被使用，LCD 的尺寸格式，将光标、增量模式、移动和闪烁等打开或关闭，然后清空显示屏。最初的十六进制代码 38x 被发送两次或更多次，以确保 LCD 模块成功进入 4 位或 8 位数据模式。然后用 0Ex 设置显示屏，06x 设置输入模式，最后用 01x 来清空显示屏。

在初始化序列之后，另一个给光标定位的命令被发送到 LCD，然后将所显示的字符发送到 LCD。如代码中所示，显示了“HELLO WORLD”。

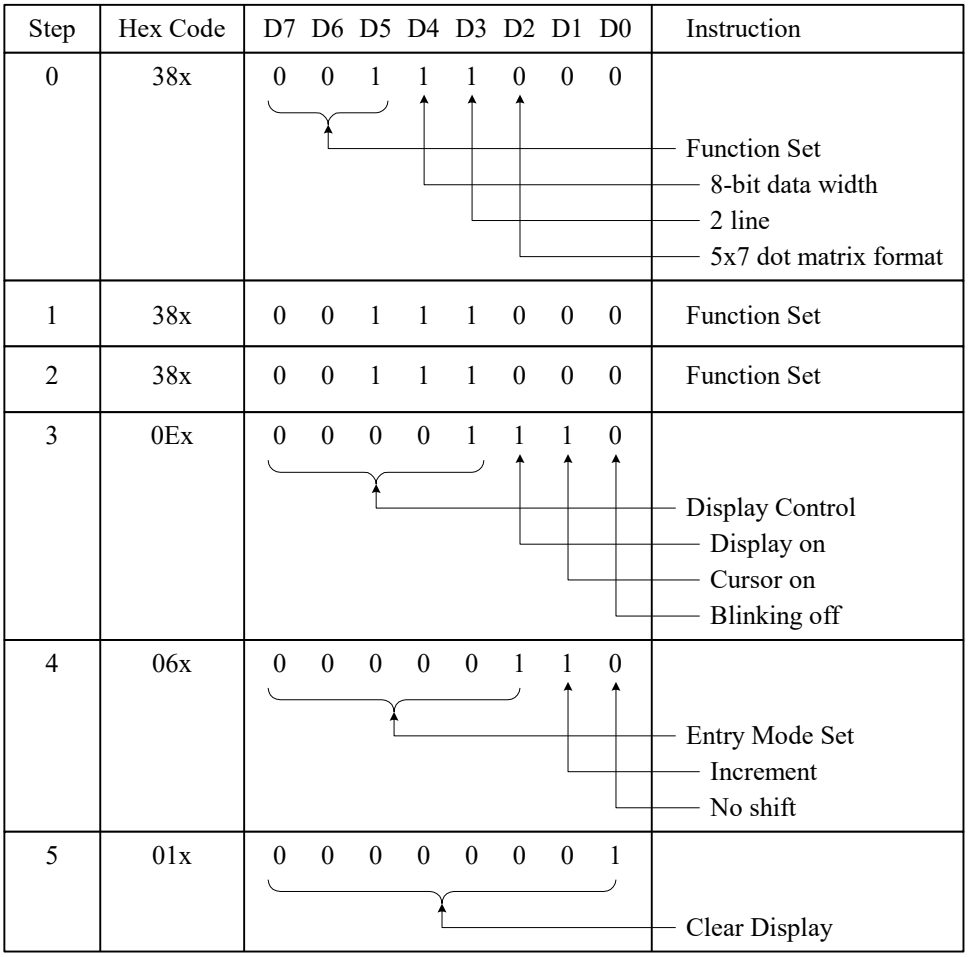


图 9.30 LCD 的初始化命令序列

9.6.2 控制器设计

控制 LCD 的专用单片机不需要数据通路。用 Verilog 和 VHDL 代码编写的 FSM 在图 9.31 和图 9.32 分别描述。FSM 状态的初始化完全按照图 9.30 所示的初始化步骤。每个十六进制码发送需要两个状态，因为在使能信号的下降沿 RS, R/W 和数据信号被锁存，所以我们每次都需要切换使能信号的状态。

```
module lcd_controller (
  input Clock, Reset,
  output reg [7:0] LCD_DATA,
  output reg LCD_RW,LCD_EN, LCD_RS,
  output LCD_ON, LCD_BLON
    );

  reg [3:0] state;
  reg [7:0] initcode[0:5];
  reg [7:0] line1[0:15];
  integer count;

  initial begin
    // LCD initialization sequence codes
```

```
  initcode[0] = 8'h38;   // Init three times
  initcode[1] = 8'h38;
  initcode[2] = 8'h38;
  initcode[3] = 8'h0E; //Display control: Display ON; Cursor ON; Blink OFF
  initcode[4] = 8'h06;   // Entry mode set: Increment One; No Shift
  initcode[5] = 8'h01;   // Display clear
  // HELLO  WORLD
  line1[0] = 8'h20;
  line1[1] = 8'h20;
  line1[2] = 8'h48;
  line1[3] = 8'h45;
  line1[4] = 8'h4C;
  line1[5] = 8'h4C;
  line1[6] = 8'h4F;
  line1[7] = 8'h20;
  line1[8] = 8'h20;
  line1[9] = 8'h57;
  line1[10] = 8'h4F;
  line1[11] = 8'h52;
  line1[12] = 8'h4C;
  line1[13] = 8'h44;
  line1[14] = 8'h20;
  line1[15] = 8'h20;
end

assign LCD_ON = 1'b1;
assign LCD_BLON = 1'b0;

always @ (posedge Clock, posedge Reset) begin
  if (Reset) begin
    count <= 0;
    state <= 4'b0000;
    end
  else
    case (state)
    // LCD initialization sequence
    // The LCD_DATA is written to the LCD at the falling edge of the E line
    // therefore we need to toggle the E line for each data write
    4'b0000: begin
      LCD_DATA <= initcode[count];
      LCD_EN <= 1'b1;  // EN=1;
      LCD_RS <= 1'b0;  // RS=0; an instruction
      LCD_RW <= 1'b0;  // R/W'=0; write
      state <= 4'b0001;
      end
    4'b0001: begin
      LCD_EN <= 1'b0;  // set EN=0;
      count <= count + 1;
      if (count+1 < 6)
        state <= 4'b0000;
      else
        state <= 4'b0010;
      end
    // move cursor to first line of display
    4'b0010: begin
      LCD_DATA <= 8'h80;  // x80 is address of 1st position on first line
    //LCD_DATA <= 8'hBF;  // xBF is address of 1st position on second line
```

```
      LCD_EN <= 1'b1;  // EN=1;
      LCD_RS <= 1'b0;  // RS=0; an instruction
      LCD_RW <= 1'b0;  // R/W'=0; write
      state <= 4'b0011;
      end
    4'b0011: begin
      LCD_EN <= 1'b0;  // EN=0; toggle EN
      count <= 0;
      state <= 4'b0100;
      end
    // write 1st line text
    4'b0100: begin
      LCD_DATA <= line1[count];
      LCD_EN <= 1'b1;  // EN=1;
      LCD_RS <= 1'b1;  // RS=1; data
      LCD_RW <= 1'b0;  // R/W'=0; write
      state <= 4'b0101;
      end
    4'b0101: begin
      LCD_EN <= 1'b0;  // EN=0; toggle EN
      count <= count + 1;
      if (count+1 < 16)
        state <= 4'b0100;
      else
        state <= 4'b1010;
      end
    4'b1010: begin
      state <= 4'b1010;
      end
    default: begin
      state <= 4'b1010;
      end
    endcase
  end // always
endmodule
```

图 9.31 16×2 LCD 控制器的 Verilog 代码

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY lcd_controller IS
PORT (
  Clock, Reset: IN STD_LOGIC;
  LCD_DATA: OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
  LCD_RW, LCD_EN, LCD_RS: OUT STD_LOGIC;
  LCD_ON, LCD_BLON: OUT STD_LOGIC);
END lcd_controller;

ARCHITECTURE FSMD OF lcd_controller IS
  TYPE state_type IS (s1,s2,s3,s4,s10,s11,s12,s13,s20,s21,s22,s23,s24);
  SIGNAL state: state_type;

  SUBTYPE ascii IS STD_LOGIC_VECTOR(7 DOWNTO 0);
  TYPE charArray IS array(0 to 15) OF ascii;
  TYPE initArray IS array(0 to 5) OF ascii;
  CONSTANT initcode: initArray := (x"38",x"38",x"38",x"0E",x"06",x"01");
```

```
  -- HELLO  WORLD
  CONSTANT line1: charArray := (x"20",x"20",x"48",x"45",x"4C",x"4C",x"4F",
                 x"20",x"20",x"57",x"4F",x"52",x"4C",x"44",x"20",x"20");
  SIGNAL count: INTEGER;

BEGIN

  LCD_ON <= '1';
  LCD_BLON <= '0';

  lcd_control: PROCESS(Clock, Reset)
  BEGIN
    IF(Reset = '1') THEN
      count <= 0;
      state <= s1;
    ELSIF(Clock'EVENT AND Clock = '1') THEN
      CASE state IS
      -- LCD initialization sequence
      -- The LCD_DATA is written to the LCD at the falling edge of the E line
      -- therefore we need to toggle the E line for each data write
      WHEN s1 =>
        LCD_DATA <= initcode(count);
        LCD_EN <= '1';   -- EN=1;
        LCD_RS <= '0';   -- RS=0; an instruction
        LCD_RW <= '0';   -- R/W'=0; write
        state <= s2;
      WHEN s2 =>
        LCD_EN <= '0';   -- set EN=0;
        count <= count + 1;
        IF count + 1 < 6 THEN
          state <= s1;
        ELSE
          state <= s10;
        END IF;

      -- move cursor to first line of display
      WHEN s10 =>
        LCD_DATA <= x"80";  -- x80 is address of 1st position on first line
      --LCD_DATA <= x"BF";  -- xBF is address of 1st position on second line
        LCD_EN <= '1';   -- EN=1;
        LCD_RS <= '0';   -- RS=0; an instruction
        LCD_RW <= '0';   -- R/W'=0; write
        state <= s11;
      WHEN s11 =>
        LCD_EN <= '0';   -- EN=0; toggle EN
        count <= 0;
        state <= s12;

      -- write 1st line text
      WHEN s12 =>
        LCD_DATA <= line1(count);
        LCD_EN <= '1';   -- EN=1;
        LCD_RS <= '1';   -- RS=1; data
        LCD_RW <= '0';   -- R/W'=0; write
        state <= s13;
      WHEN s13 =>
        LCD_EN <= '0';   -- EN=0; toggle EN
```

```
        count <= count + 1;
        IF count + 1 < 16 THEN
          state <= s12;
        ELSE
          state <= s20;
        END IF;
      WHEN s20 =>
        state <= s20;
      WHEN OTHERS =>
        state <= s20;
      END CASE;
    END IF;
  END PROCESS;
END FSMD;
```

图 9.32 16×2 LCD 控制器的 VHDL 代码

9.7 VGA 显示器控制器

在这一小节中我们会设计和实现用来控制 VGA 显示器的微型控制器。这个控制器可以控制 VGA 显示器显示简单的动态图像。

9.7.1 工作原理

低分辨率的 VGA 显示器一般包含 640 列×480 行的图像元素，也叫作**像素**。高分辨率的显示器有更多的行与列，但是主要思想还是一样的。一个图片的显示是通过点亮或者熄灭每个独立的像素来实现的。尽管点亮一个像素看上去没有什么区别，但是当一大片像素被同时点亮的时候。那么这个像素组合就可以显示衣服图像。控制器不停地扫描整个屏幕，以非常快的速度来点亮或者熄灭像素点。尽管只有每次扫描只有一个像素点被点亮，但是因为控制器扫描的速度非常快，由于视觉残留的作用，你可以看到整幅图像。

图 9.33 显示了扫描从左上角的第 0 行第 0 列开始，然后一直右移直到结束为止。当扫描到最后，它开始下一行。当扫描进行到最右下角的那个像素点的时候，它再次回到最左上角的那个像素点。为了减小屏幕上的频闪，整个屏幕的扫描速度都必须在每秒 60 次及以上。在横向和纵向的重描中，所有像素点都被熄灭。

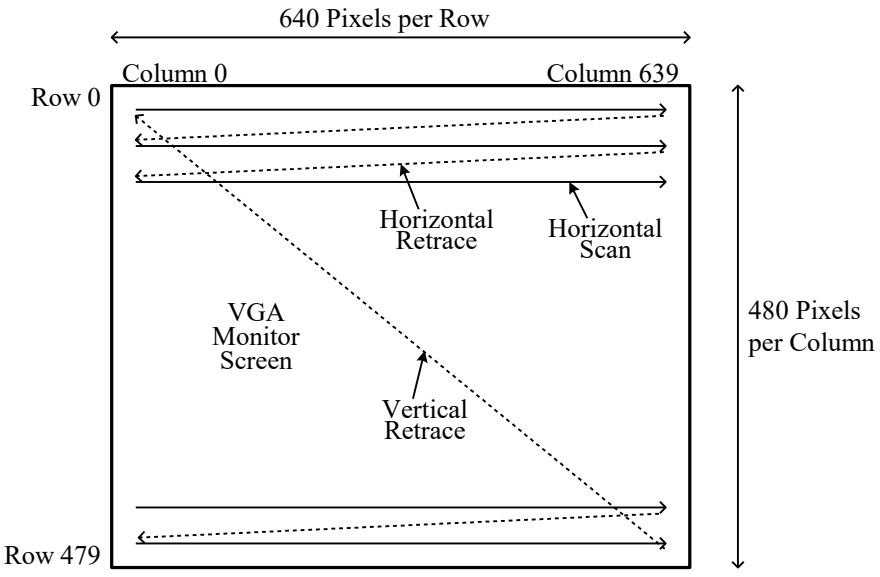


图 9.33 640 列×480 行的 VGA 显示器 扫描从第 0 行第 0 列开始不停往右往下倒第 639 行第 479 列

VGA 显示器被五个信号控制：红、绿、蓝、横向同步、纵向同步。三个颜色信号代表了某个像素点的 RGB 信号。三个信号是独立的，所以一共可以显示 8 种颜色。新的 VGA 显示器用更多的比特显示颜色，从而获得更多的颜色组合。横向和纵向的同步信号用来控制扫描速度。横向同步信号决定了横向扫描的时间，纵向同理。通过控制这五个信号，显示器可以显示画面。

横向和纵向同步信号的时间图显示在了图 9.34 中。当不活动时，两个同步信号都是 1。行的扫描开始于横向同步信号变成低电平持续 3.77usec 的时间，如图 9.34 中的段落 B。然后是 1.79usec 的高电平时间，u 图中的段落 C。接下来控制 640 列的三个颜色信号的数据被发送，与此同时行信号保持高电平 25.42usec，如图所示的段落 D。最后，在最后一列像素扫描完之后，有额外的一个 0.79usec 时间全部的 RGB 信号是空状态的，如图中段落 E 所示。然后横向同步信号再次变为低电平为了下一行的扫描。总共扫描一行的时间为 31.77usec。

纵向同步信号的时间与横向同步信号的时间类似。64usec 的高低纵向同步信号可以使得复位屏幕的左上角，如图中 P 段落。接下来是 1020usec 的高电平信号在图中的 Q 段落。再接下来是每行 31.77usec 的横扫描时间，总共为 15250usec，如段落 R 所示。最后，在扫描完最后一行之后，有额外的 450usec 时间，如段落 S 所示，紧跟着的是纵向的同步信号再次变成低电平去出发另一个完整的屏幕扫描周期。完成一次完整的扫描的事件是 16784usec。

为了显示器工作正常，我们需要让横向和纵向的同步信号准确，然后发送 TGB 数据到每个指定行列位置的像素点。我们发现，如果我们使用 25.175MHz 的同步时钟频率来校准两个同步信号会非常的简单。（更高的分辨率显示器需要更快的时钟频率信号）对于 25.175MHz 的时钟信号频率，他的周期是 $1 / 25.175 \times 10^6$ ，大约是 0.0397usec 每个时钟周期。对于段落 B 里的横向同步信号，我们需要 3.77usec，大约是 3.77 / 0.0397 = 95 个时钟周期。对于段落 C，我们需要 1.79usec，大约是 45 个时钟周期。相似的，我们需要 640 个时钟周期给段落 D 的 640 列像素，和 20 个时钟周期给段落 E。总共完成一次行扫描的时间是 800 个时钟周期。注意到，对于 25.175MHz 的时钟来说，段落 D 需要正好 640 个时钟周期，对应着每行的 640 列。所以，对于分辨率越高的显示器，我们需要频率更高的时钟。

纵向时间是横向时钟周期的倍数。举例来说，段落 P 是 64usec，大约是（2 × 31.77），然而段落 Q 是 32 （1020 / 31.77） 个时钟周期。段落 R 则是 15250usec / 31.77 usec = 480 。 当然，它需要正好 480 次，因为我们整个屏幕有 480 行。最后，段落 S 还需要 14 个周期。

为了产生正确的横向和纵向的同步信号，电路需要跟踪扫描时候的段落 D 中的当前列数和段落 R 中的当前行数，目的是为了知道什么时候去点亮或者熄灭这个像素点。举例来说，为了使得某一个像素点变成绿色，需要检测现在的行数和列数。如果他们正好等于你需要点亮绿色的那个像素点的行列数，那么你就写入绿色信号，然后这个像素点就会变成绿色。

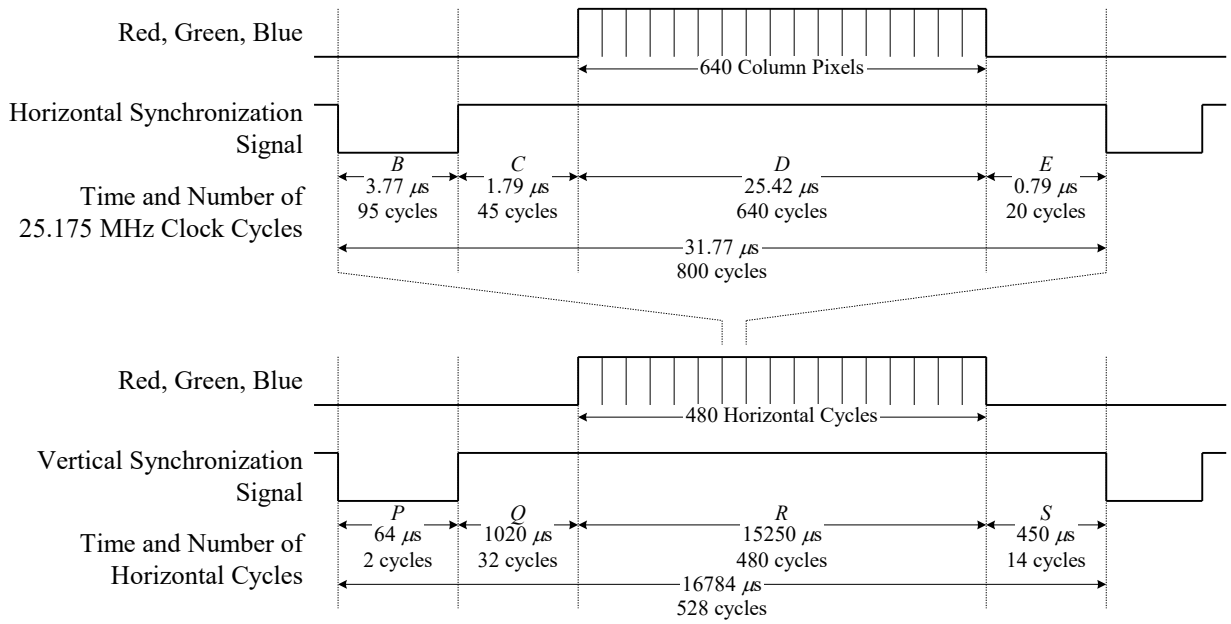


图 9.34 横向和纵向同步信号的时间图

9.7.2 控制器设计

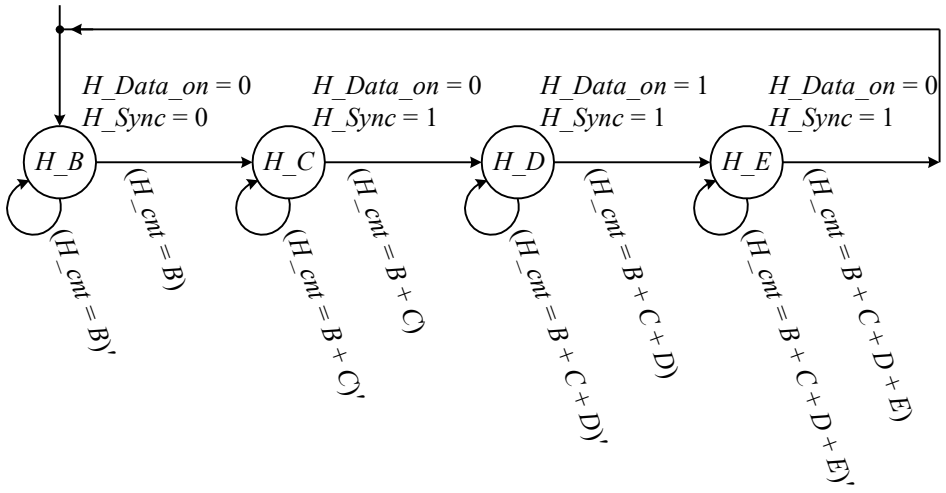
为了得到正确的横向和纵向同步信号，我们需要设计一个运作在 25.175MHz 的时钟信号的拥有 800 个状态的有限状态机。在最初的 95 个状态里，横向同步信号 *H_Sync* 置为 0。在接下来的 45 + 640 + 20 = =705 个状态中，*H_Sync* 置为 1。问题在于，人工地产生一个 800 个状态有限状态机的电路非常地困难。一个简单办法是只使用两个状态：一个是 H_Sync 在段落 B 内置 0，另一个是在段落 C, D, E 内置 1。我们然后使用一个使用相同时钟频率的计数器来测量这个有限状态机到底在这个状态里循环了多少次。在第一个状态里，我们需要停留 95 次才能进入下一个状态，在第二个状态里，我们需要停留 705 次才能回到第一个状态。在第一个状态里，*H_Sync* 置为 0；第二个状态里，*H_Sync* 置为 1。

为了帮助跟踪三个颜色信号使能的时间，我们需要产生一个 *H_Data_on* 的信号，产生方法与产生 *H_Sync* 是一样的。*H_Data_on* 在段落内有效，在段落 *B, C, E* 中无效。因此，对于段落 D 中的重复的 640 的周期中，*H_Data_on* 置 1，然后段落 B, C, E 中的重复的 160 个周期中置 0。

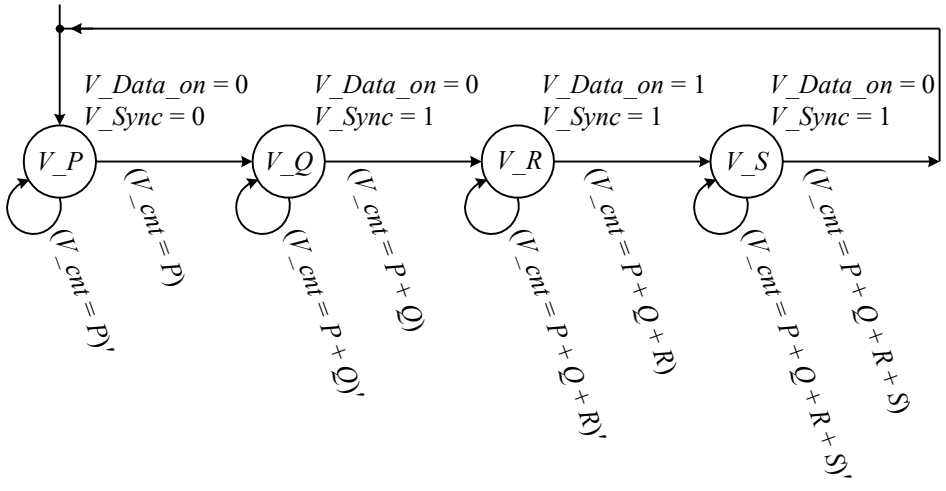
结合起 *H_Sync* 的两个状态和 *H_Data_on* 的两个状态，我们最终得到了状态图，如图 9.35(a)。这个有限状态机有分别对于段落 *B, C, D, E* 的四个段落的四个状态。计数器一开始为 0，然后在每个时钟周期加 1。在段落 B 的状态 *H_B* 中，有限状态机输出 0 给 *H_Data_on* 和 *H_Sync*。然后停在状态 *H_B*95 个时钟周期。条件（*H_cnt = B*）为了检查计数器是否等于 *B*，其中 *B* 等于 95。当计数器到 95 的时候，优先状态机转移到状态 *H_C*，对于着段落 *C*。在状态 *H_C* 中，有限状态机输出 0 给 *H_Data_on* 输出 1 给 *H_Sync*，然后停留 45 个时钟周期（i.e. 直到 *H_cnt = B + C = 95 + 45 = 140*）。当 *H_cnt* 得到 140 时候，有限状态机转向状态 *H_D*，然后输出 1 给 *H_Data_on* 和 *H_Sync*。当 *H_cnt = B + C + D = 95 + 45 + 640 = 780* 的时候，有限状态机转移到状态 *H_E*，然后输出 1 给 *H_Data_on* 输出 0 给 *H_Sync* 。当有限状态机在状态 *H_E* 里面呆了 20 个时钟周期直到 *H_cnt = B + C + D + E = 95 + 45 + 640 + 20 = 800* 时，它回溯到了状态 *H_B*。当有限状态机回溯到状态 *H_B* 的时候，*H_cnt* 被 reset 为 0，然后开始下一行的扫描。

纵向同步信号与横向同步信号相类似，我们可以建立类似的第二个计数器和第二个有限状态机。这个第二个纵向的有限状态机与横向的那个一样。唯一的不同是在于时钟频率。观察图 9.34 中纵向同步信号里的每个段落，我们看到段落 P 的 64usec 几乎是整个横向扫描信号 31.77usec 的两倍。段落 Q 的 1020usec 大约是 32 个横向扫描信号（1020 / 31.77 约等于 32）。对于段落 R，则是 480 个横向扫描周期，对于段落 S，则是 14 个横向扫描 zhouqi.yinci 纵向计数器和有限状态机的时钟可以从横向的计数器中得到。横向计数器每计数 800 次，纵向计数器就加 1。

图 9.35(c)和(d)显示了横向纵向有限状态机的状态转移图。注意到除了四个计数条件，其他都是一样的。所以我们可以用相同的电路来表示有限状态机。唯一的不同在于他们给四个计数条件的输入状态信号来自于不同的计数比较器。图 9.35(e)显示输出表对于两个 FSM 也是相同的。只需要产生两个输出信号 *H_Data_on* 和 *H_Sync* 给横向的 FSM 和 *V_Data_on* 和 *V_Sync* 给纵向的 FSM。最终有限状态机电路如图 9.35(f)所示。我们需要利用两个这个 FSM 电路的例子：一个横向 FSM 和一个纵向 FSM。给横向 FSM 的时钟信号的 25.175MHz，纵向 FSM 的时钟是从横向 FSM 计数器的 *Rollover* 信号中提取的。



(a)



(b)

当前状态 Q_1Q_0		次态（实现） $Q_{1next}Q_{0next}$ (D_1D_0)							
		$(H_cnt = B)$		$(H_cnt = B + C)$		$(H_cnt = B + C + D)$		$(H_cnt = B + C + D + E)$	
		0	1	0	1	0	1	0	1
00	*H_B*	00	01	—	—	—	—	—	—
01	*H_C*	—	—	01	10	—	—	—	—
10	*H_D*	—	—	—	—	10	11	—	—
11	*H_E*	—	—	—	—	—	—	11	00

$Q_{1next} = Q_1'Q_0\,(H_cnt = B + C) + Q_1Q_0' + Q_1Q_0\,(H_cnt = B + C + D + E)'$

$Q_{0next} = Q_1'Q_0'\,(H_cnt = B) + Q_1'Q_0\,(H_cnt = B + C)' + Q_1Q_0'\,(H_cnt = B + C + D) + Q_1Q_0\,(H_cnt = B + C + D + E)'$

(c)

当前状态 Q_1Q_0		次态（实现） $Q_{1next}Q_{0next}$ (D_1D_0)							
		$(V_cnt = P)$		$(V_cnt = P + Q)$		$(V_cnt = P + Q + R)$		$(V_cnt = P + Q + R + S)$	
		0	1	0	1	0	1	0	1
00	*V_P*	00	01	—	—	—	—	—	—
01	*V_Q*	—	—	01	10	—	—	—	—
10	*V_R*	—	—	—	—	10	11	—	—
11	*V_S*	—	—	—	—	—	—	11	00

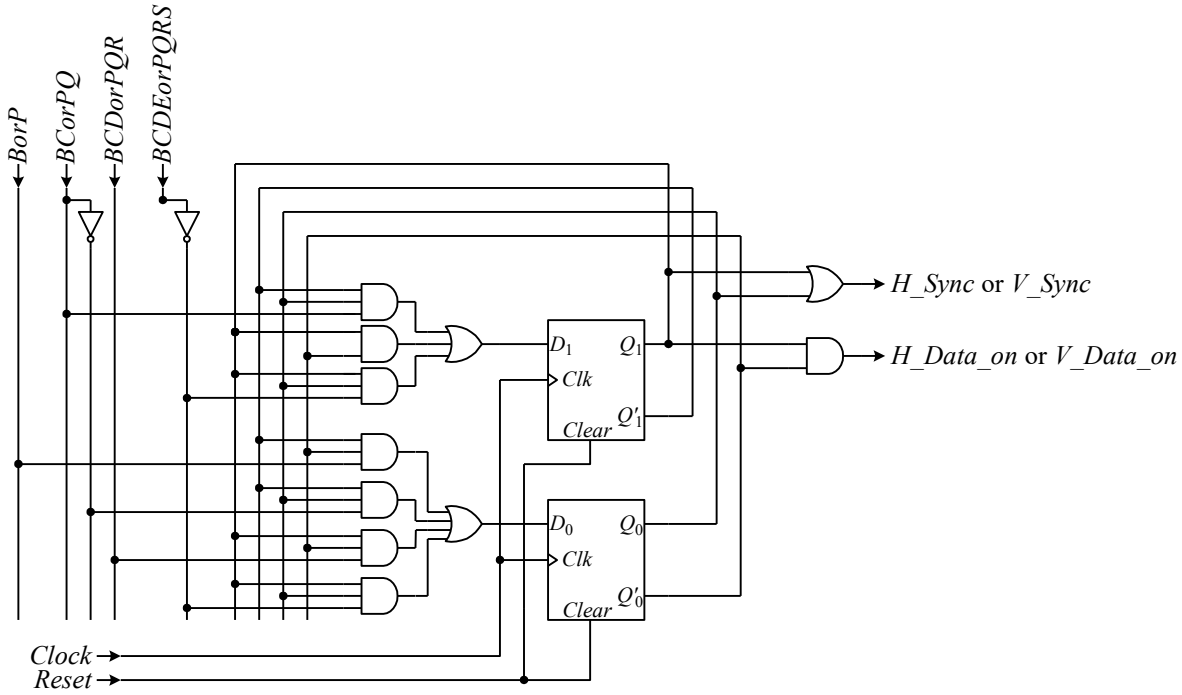
$Q_{1next} = Q_1'Q_0\,(V_cnt = P + Q) + Q_1Q_0' + Q_1Q_0\,(V_cnt = P + Q + R + S)'$

$Q_{0next} = Q_1'Q_0'\,(V_cnt = P) + Q_1'Q_0\,(V_cnt = P + Q)' + Q_1Q_0'\,(V_cnt = P + Q + R) + Q_1Q_0\,(V_cnt = P + Q + R + S)'$

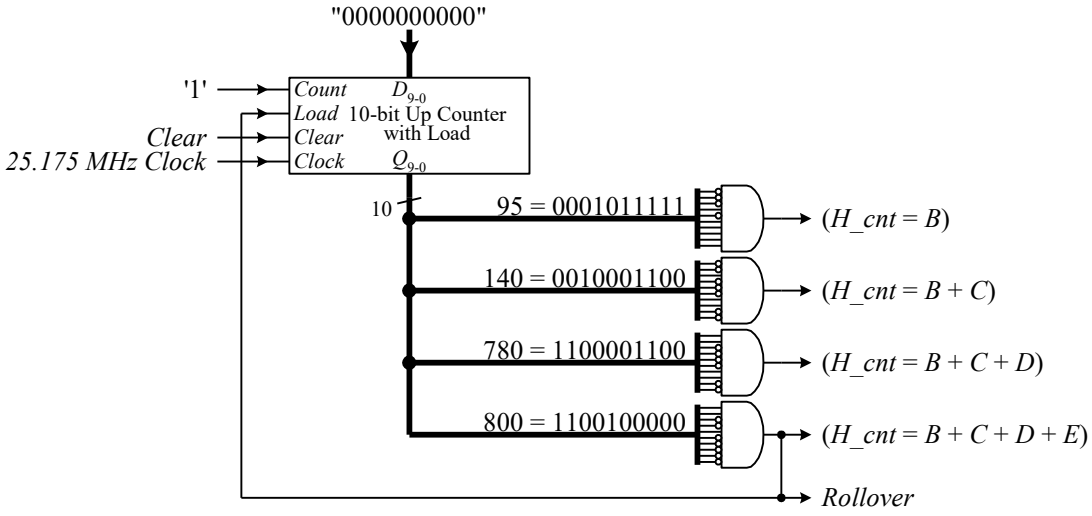
(d)

当前状态 Q_1Q_0	H_Data_on or *V_Data_on*	H_Sync or V_Sync
00	0	0
01	0	1
10	1	1
11	0	1

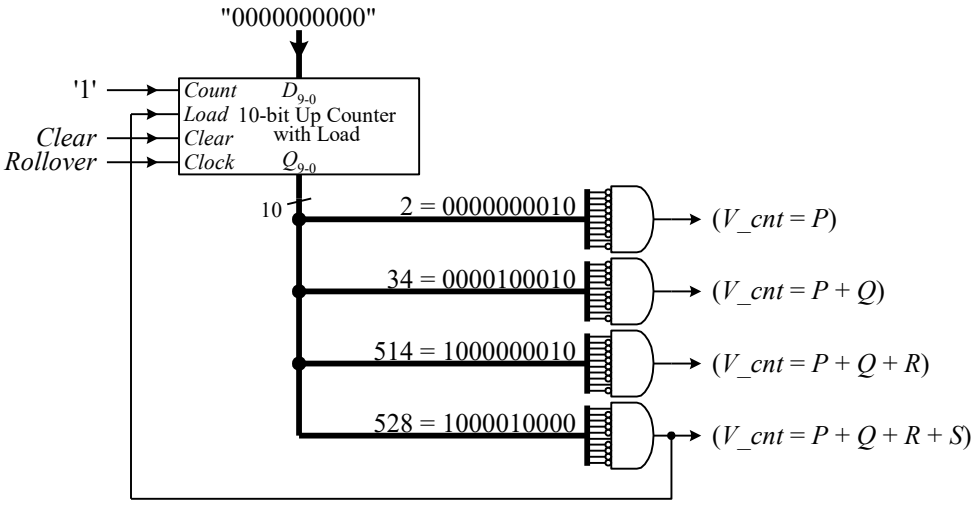
(e)


BorP
BCorPQ
BCDorPQR
BCDEorPQRS
H_Sync or V_Sync
H_Data_on or V_Data_on
D1
Q1
Clk
Clear
Q'1
D0
Q0
Clk
Clear
Q'0
Clock
Reset

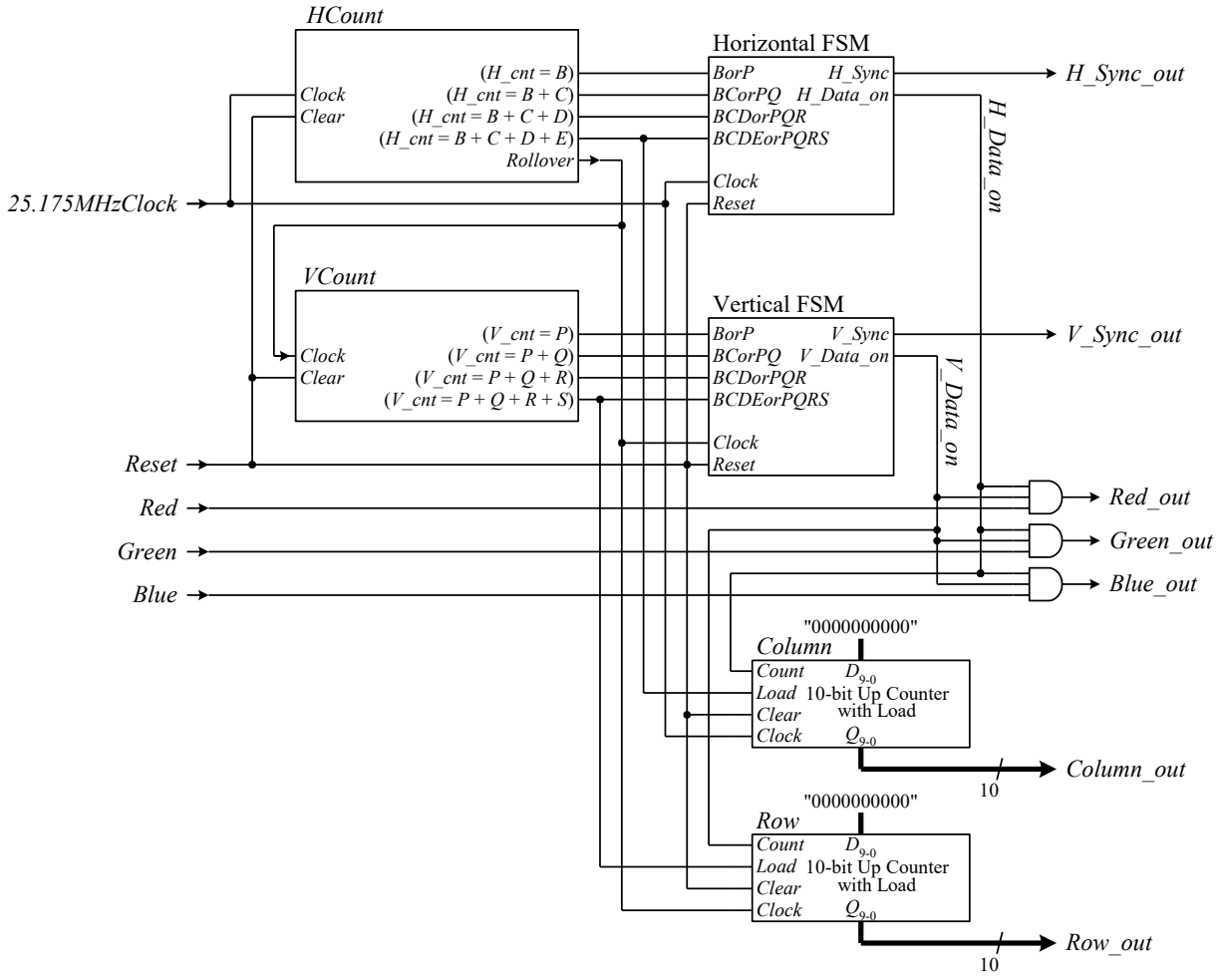
(f)


"0000000000"
'1'
Clear
25.175 MHz Clock
Count
Load
Clear
Clock
D9-0
10-bit Up Counter
with Load
Q9-0
10
95 = 0001011111
140 = 0010001100
780 = 1100001100
800 = 1100100000
(H_cnt = B)
(H_cnt = B + C)
(H_cnt = B + C + D)
(H_cnt = B + C + D + E)
Rollover

(g)


"0000000000"
'1'
Clear
Rollover
Count
Load
Clear
Clock
D_{9-0}
10-bit Up Counter
with Load
Q_{9-0}
10
2 = 0000000010
34 = 0000100010
514 = 1000000010
528 = 1000010000
(V_cnt = P)
(V_cnt = P + Q)
(V_cnt = P + Q + R)
(V_cnt = P + Q + R + S)

(h)


HCount
Clock
Clear
(H_cnt = B)
(H_cnt = B + C)
(H_cnt = B + C + D)
(H_cnt = B + C + D + E)
Rollover
Horizontal FSM
BorP
BCorPQ
BCDorPQR
BCDEorPQRS
Clock
Reset
H_Sync
H_Data_on
H_Sync_out
H_Data_on
25.175MHzClock
VCount
(V_cnt = P)
(V_cnt = P + Q)
(V_cnt = P + Q + R)
(V_cnt = P + Q + R + S)
Vertical FSM
V_Sync
V_Data_on
V_Sync_out
V_Data_on
Reset
Red
Green
Blue
Red_out
Green_out
Blue_out
"0000000000"
Column
Count
Load
Clear
Clock
D_{9-0}
10-bit Up Counter
with Load
Q_{9-0}
10
Column_out
"0000000000"
Row
Row_out

(i)

图 9.35 VGA 显示器的控制器：(a)横向同步的状态图(b)纵向同步的状态图(c)横向同步的状态转移图(d)纵向同步的状态转移图(e)输出表(f)横向和纵向同步的 FSM 电路(g)横向同步计数器(h)纵向同步计数器(i)整个 VGA 控制器的完整电路

给四个 FSM，*BorP, BCorPQ, BCDorPQR* 和 *BCDEorPQRS* 的状态信号是从两个计数器产生的。一个横向计数器和一个纵向计数器。横向计数器 HCount，带有四个比较器 (*H_cnt = B), (H_cnt = B + C), (H_cnt =B + C +D)*和（*H_cnt = B + C + D + E*）如图 9.35(g)。从 0 数到 800 需要一个 10 个 bit 的计数器。因此每个比较器都带有一个 10 输入的与门。每个与门的输入为 *B = 95, B + C = 95 + 45 =140, B + C + D = 95 + 45 + 640 = 780* 和 *B + C + D + E = 95 + 45 + 640 + 20 =800* . 通过 Load 信号线有效来使得计数器周期回到 0，当计数器达到了 800 以后，然后使得计数器清零。横向计数器的时钟信号为 25.175MHz。比较器的输出（*H_cnt = 800*）是 *Rollover* 信号，然后被用作纵向计数器、纵向 FSM 和行计数器的时钟信号。

纵向计数器，*VCount*， 带有四个比较器(*V_cnt = P*) , (*V_cnt = P + Q*), (*V_cnt = P + Q + R*)和(*V_cnt = P + Q + R + S*)，如图 9.35(b)。这个计数器的电路与横向计数器的一样，除了比较器的值不一样，我们不需要输出一个 *rollover* 时钟信号。这个计数器的时钟是来自于横向计数器的 *Rollover* 信号。

整个 VGA 显示器控制器电路如图 9.35(i)所示。为了使得三个 RGB 颜色信号有效，它们必须只在段落 D 和 R 中被点亮。所以，三个颜色输出信号 *Red_out, Green_out, Blue_out* 需要与 *H_Data_on* 和 *V_Data_on* 两个信号作『与』操作。比如，如果输入 *Red* 信号为 1，那么只有扫描在段落 D 和 R 中的时候，输出 *Red_out* 才会是 1。

最后，为了点亮某一个像素点，电路需要跟踪扫描过程中的段落 D 的当前列和段落 R 的当前行。两个额外的计数器，*Column* 和 *Row* 就是作此用途。因为他们需要相应地在扫描到段落 D 和 R 时候计数，因此，*Column* 计数器 *Count* 的输入是由信号 *H_Data_on* 来决定的，同理，*Row* 计数器 *Count* 的输入是由信号 *V_Data_on* 来决定的。一旦计数器分别达到了 640 和 480，那么他们就已经经过了这两个段落，因此计数器不会再继续计数。他们会在扫描再次经过这个段落之前清零。在电路中，两个 *Load* 信号有效使得两个计数器清零。最终，*Column* 计数器的时钟来自 25.175MHz 的信号源，而 *Row* 计数器的时钟提取自横向计数器。

9.7.3 控制器实现

为了在屏幕上显示图像，你需要扫描到你需要点亮的像素点的行列值，然后输入 RGB 值。距离，如果你只要使得 *Red* 信号连续有效，所有的像素点都将会是红色，然后你就会看到整个屏幕都变成红色。另外，如果你只是想要第一行像素变成红色，你需要使得 *Red* 信号有效当 *Row* 计数器为 0 。为了得到整个边框都是红色的，你需要使得 *Red* 信号有效当 *Row* = 0 ，*Row* = 639，*Column* = 0，*Column* = 479。图 9.36 显示了整个电路利用 VGA 控制器电路使得整个屏幕的边框为红色，如图 9.35(i)。

一般情况下，会有一个二维的视频内存空间来存储每一个像素点的 RGB 值。行列计数器是用来指示视频内存空间中的地址的。每个地址对应内存里的值是需要显示的 RGB 值。

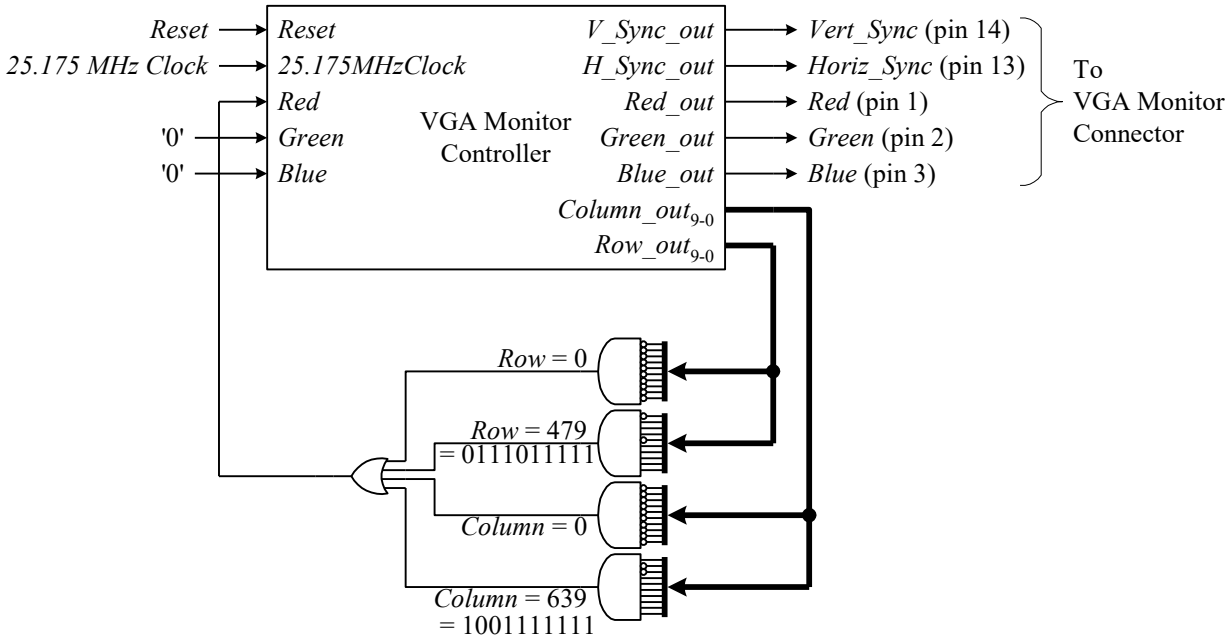


图 9.36 整个电路利用 VGA 控制器电路使得整个屏幕的边框为红色

为了是的每个像素可以显示更多的颜色，三个颜色信号需要用更多 bit。举例来说，在 Altera DE1 FPGA 开发板上，三种颜色的每种都用四个 bit 来表示。在 DE2 开发板上，每个颜色用十个 bit 来表示。在 Xilinx Spartan 3E 开发板上，每个颜色只有一个 bit。在 XUP Virtex - II Pro 开发板上，每个颜色有八个 bit。不管每个颜色用了多少比特，总是有三条线镰刀显示器因为这三个 RGB 信号实际上是模拟信号。所以在控制器的输出端和 VGA 显示器的输入端之间需要有数模转换器。

对于分辨率更高的显示器，则需要有更快的时钟速度。下表显示了另外两种分辨率的 VGA 显示器的时钟要求。

9.8 用于温度传感器的 A/D 控制器

众多感知设备(例如光传感器、温度传感器、压力传感器等)输出的是模拟信号，而非数字信号。为了使微处理器能够处理模拟信号，需要通过 A/D 转换器将模拟信号转换为数字信号。在本节中，我们将设计一个包含 A/D 转换器接口的微控制器，用于读取温度传感器的模拟数值。

A/D 转换芯片的种类众多，且拥有不同的功能、精度、封装及接口。我们采用美国国家半导体公司生产的 ADC0832 A/D 转换器芯片，它是一个 8 比特 A/D 转换芯片，通过串行数据链路与微控制器相连接。正因为如此，A/D 转换器能够定位正确的模拟信号源，并能够通过少量连接线与微处理器进行通信，且串行比特数据流受噪声的影响极低。这些特性极大地简化了保持模拟信号准确性的电路结构，否则该电路将极易受噪声影响。

9.8.1 工作原理

ADC0832 8 比特 A/D 转换芯片通过串行数据链路与微控制器相连接，微控制器与 ADC 芯片之间的通信需要三或四根线与之相连。在我们的例子中，我们将采用四根线，其中三根线为输入信号：时钟(CLK)，芯片选择(CSN)和数据输入(DI)；另一根为输出信号：数据输出(DO)。

低电平有效的 CSN 信号线用于使能 ADC 芯片，控制器命令通过 DI 信号线发送至 ADC，被 ADC 所转换的数字数据将通过 DO 信号线发送至控制器，时钟信号用于同步 DI 和 DO 线上的数据传输。微控制器和 ADC 芯片之间的连接结构如图 9.37 所示。

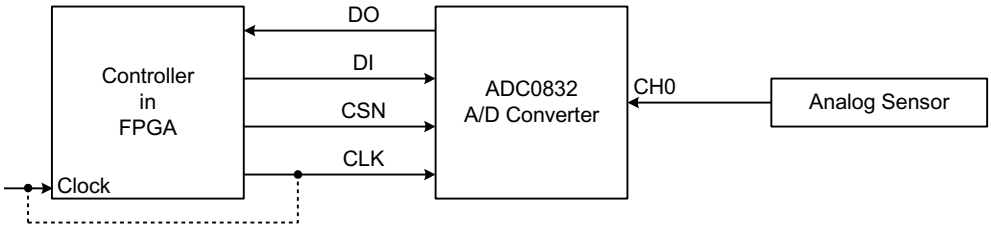


图 9.37 基于 FPGA 的微控制器与 A/D 转换芯片间的串行数据链路连接示意图

模数转换过程的初始化可通过将芯片选择(SCN)线设定为低电平来实现，且 CSN 线在整个转换过程中均需保持低电平状态。随后，ADC 芯片等待起始位和命令。接着，微控制器在时钟(CLK)信号线上生成时钟信号（在没有外部持续提供的情况下，如图 9.37 中虚线连接所示）。一个 1 起始位和一个 2 比特 MUX 地址命令在每一个时钟上升沿通过输入数据(DI)线被发送至 ADC。首个 MUX 地址命令位用于选择单端或差分模拟输入。一个 1 比特用于选取单端选项，单端输入采用接地作为参考输入。在选择了单端选项之后，第二个比特用来选取使用芯片上的哪两个输入信道。一个 0 比特用于选择芯片上的 0 信道。数据输出(DO)线则输出三态和转换后的数字数据（以最重要的比特为开头），并通过这条线将其发送至微控制器。随后，相同的转换后的数字数据（不过以最不重要的比特为开头）通过 DO 线将其发送至微控制器。在数据传输过程的结尾，CSN 线需要在开始进行下一个转换过程前声明无效。微控制器与 A/D 转换芯片间通信的时序图如图 9.38 所示。

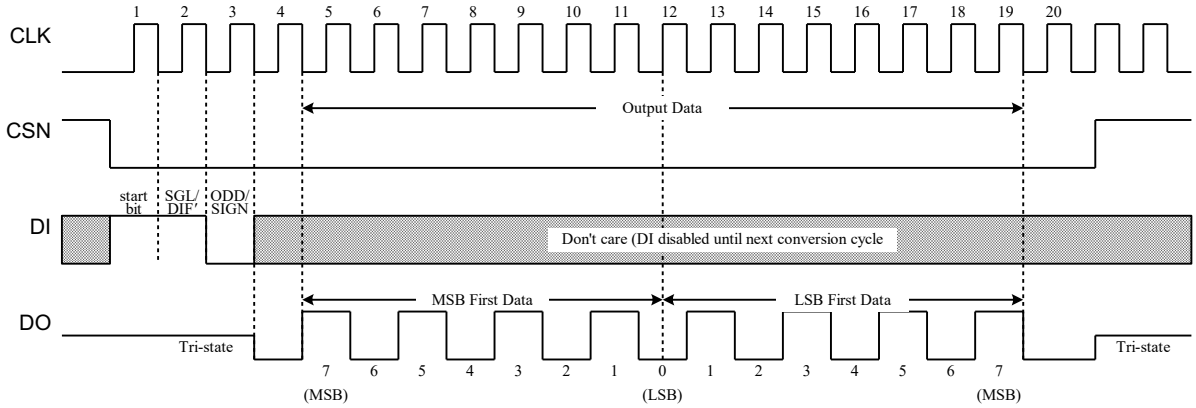


图 9.38 微控制器与 A/D 转换芯片间通信的时序图

9.8.2 控制器设计

我们将利用 FSMD 模型编写 HDL 代码，以描述 A/D 转换芯片的控制器。图 9.40 展示了该 FSM 控制器的 Verilog 代码。状态转换和由 FSM 代码生成的输出信号与前述图 9.38 所示的时序图非常接近。

FSM 的输入时钟速率并不是至关重要的。FSM 通过在每个状态转换时固定输出时钟(CLK)以生成输出时钟(CLK)信号，从而同步其与 ADC 芯片之间的数据传输。

```
module ADC_FSM
(
  input Clock, Reset,
  input DO,
  output reg CLK, CSN, DI,
  output reg [7:0] DataOut,
  output reg DataReady
);
  reg [7:0] MSBF_Buffer, LSBF_Buffer;
  reg [2:0] index;
  reg [3:0] state;
```

```
always @ (posedge Clock or posedge Reset) begin
 if (Reset) begin
  CSN <= 1;
  CLK <= 0;
  DI <= 1; // don't care
  DataReady <= 0;
  state <= 4'b0000;
 end else begin
  case (state)
   4'b0000: begin
    CSN <= 0;
    CLK <= 0;
    DI <= 1; // start bit
    DataReady <= 0;
    state <= 4'b0001;
    end
   4'b0001: begin
    CSN <= 0;
    CLK <= 1;
    DI <= 1;
    DataReady <= 0;
    state <= 4'b0010;
    end
   4'b0010: begin
    CLK <= 0;
    DI <= 1; // set SGL/DIF'
    state <= 4'b0011;
    end
   4'b0011: begin
    CLK <= 1;
    state <= 4'b0100;
    end
   4'b0100: begin
    CLK <= 0;
    DI <= 0; // set ODD/SGN'; 0 = channel 0, 1 = channel 1
    state <= 4'b0101;
    end
   4'b0101: begin
    CLK <= 1;
    state <= 4'b0110;
    end
   4'b0110: begin
    CLK <= 0;  // MUX settling time
    state <= 4'b0111;
    end
   4'b0111: begin
    CLK <= 1;
    index <= 7;
    state <= 4'b1000;
    end
   4'b1000: begin
    CLK <= 0;
    state <= 4'b1001;
    end
   4'b1001: begin
    CLK <= 1;
```

```
        MSBF_Buffer[index] <= DO;        // receive 8 data bits, MSB first
        if (index > 0) begin
          index <= index - 1;
          state <= 4'b1000;
        end else begin
          LSBF_Buffer[index] <= DO;  // receive 8 data bits, LSB first
          index <= index + 1;
          state <= 4'b1010;
          end
        end
       4'b1010: begin
        CLK <= 0;
        state <= 4'b1011;
        end
       4'b1011: begin
        CLK <= 1;
        LSBF_Buffer[index] <= DO;
        if (index < 7) begin
          index <= index + 1;
          state <= 4'b1010;
        end else
          state <= 4'b1100;
        end
       4'b1100: begin
        CLK <= 1;
        CSN <= 1;
        if (MSBF_Buffer == LSBF_Buffer) begin
          DataOut <= MSBF_Buffer;
          DataReady <= 1;
        end else begin
          DataOut <= 8'b00000000;
          DataReady <= 0;
          end
        state <= 4'b0000;
        end
       default: begin
        state <= 4'b0000;
       end
     endcase
    end // else
  end // always
endmodule
```

图 9.39 A/D 转换控制器的行为级 Verilog 代码

图 9.40 为 A/D 转换控制器的 VHDL 代码。它与之前的 Verilog 代码非常相似，只不过它采用相同的外部时钟信号来同时驱动 FSM 和 ADC 芯片，而非利用 FSM 来生成一个单独的同步 CLK 信号。因此，该 FSM 不需要输出第二个 CLK 信号。该 FSM 的时钟连接如图 9.37 中虚线所示。命令的发送和转换后数据的接收均在时钟信号的下降沿完成。

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY ADC_FSM IS PORT (
    Clock   : IN STD_LOGIC;
    Reset   : IN STD_LOGIC;
    DO      : IN STD_LOGIC;
```

```
    CSN     : OUT STD_LOGIC;
    DI      : OUT STD_LOGIC;
    DataOut : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
    DataReady  : OUT STD_LOGIC
  );
END ENTITY;

ARCHITECTURE FSM OF ADC_FSM IS
  TYPE state_type IS (s0, s1, s2, s3, s4, s5, s6, s7);
  SIGNAL state : state_type;

  SIGNAL MSBF_Buffer: STD_LOGIC_VECTOR(7 DOWNTO 0);
  SIGNAL LSBF_Buffer: STD_LOGIC_VECTOR(7 DOWNTO 0);
  SIGNAL index: INTEGER;

BEGIN
  PROCESS (Clock, Reset)
  BEGIN
    IF Reset = '1' THEN
      CSN <= '1';
      DataReady <= '0';
      index <= 0;
      state <= s0;
    ELSIF Clock'EVENT AND Clock = '0' THEN
      CASE state IS
        WHEN s0 =>
          CSN <= '0';
          DI <= '1'; -- start bit
          DataReady <= '0';
          state <= s1;
        WHEN s1 =>
          CSN <= '0';
          DI <= '1'; -- set SGL/DIF'
          DataReady <= '0';
          state <= s2;
        WHEN s2 =>
          CSN <= '0';
          DI <= '0'; -- set ODD/SIGN; 0 = channel 0, 1 = channel 1
          DataReady <= '0';
          state <= s3;
        WHEN s3 => -- wait for one clock for Mux setting
          CSN <= '0';
          DataReady <= '0';
          state <= s4;
        WHEN s4=> -- wait for one clock for Mux setting
          CSN <= '0';
          DataReady <= '0';
          index <= 7;
          state <= s5;
        WHEN s5 =>
          CSN <= '0';
          DataReady <= '0';
          IF (index > 0) THEN
            MSBF_Buffer(index) <= DO;
            index <= index - 1;
            state <= s5;
          ELSE
```

```
          MSBF_Buffer(index)  <= DO;
          LSBF_Buffer(index)  <= DO;
          index <= 1;
          state <= s6;
        END IF ;
       WHEN s6 =>
        CSN <= '0';
        DataReady <= '0';
        IF (index <= 7) THEN
          LSBF_Buffer(index)  <= DO;
          index <= index + 1;
          state <= s6;
        ELSE
          state <= s7;
        END IF ;
       WHEN s7 =>
        CSN <= '1';
        DataReady <= '1';

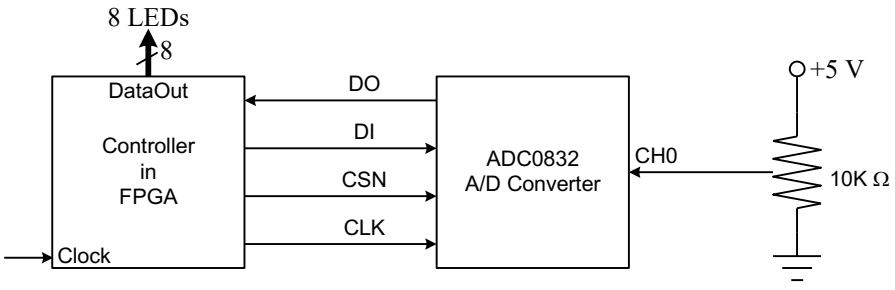
        DataOut(0) <= MSBF_Buffer(0);
        DataOut(1) <= MSBF_Buffer(1);
        DataOut(2) <= MSBF_Buffer(2);
        DataOut(3) <= MSBF_Buffer(3);
        DataOut(4) <= MSBF_Buffer(4);
        DataOut(5) <= MSBF_Buffer(5);
        DataOut(6) <= MSBF_Buffer(6);
        DataOut(7) <= MSBF_Buffer(7);
        state <= s0;
       WHEN OTHERS =>
     END case;
    END IF ;
  END PROCESS;
END FSM;
```

图 9.40 A/D 转换控制器的行为级 VHDL 代码

9.8.3 应用实现

一个简单的测试 FSM 控制器代码的方法是将一个可调式 10 千欧电阻器与 ADC 芯片的 0 信道模拟输入端相连，如图 9.41(a)所示。电位计的一端接在+5V 电源上，另一端接地。电位计的中间端与 ADC 芯片的 0 信道相连。由于 ADC0832 是一个 8 比特 A/D 转换芯片，因此输出数字量的范围为 0~255。当电位计转向一侧，其输出值应为 0；而当电位计转向另一侧，其输出值应为 255。

ADC 芯片的输入可采用不同的模拟传感器。图 9.41(b)展示了热敏电阻式温度传感器的连接示意图，而图 9.41(c)展示了压力传感器的连接示意图。该压力传感器需要一个运算放大器将信号进行放大。



(a)

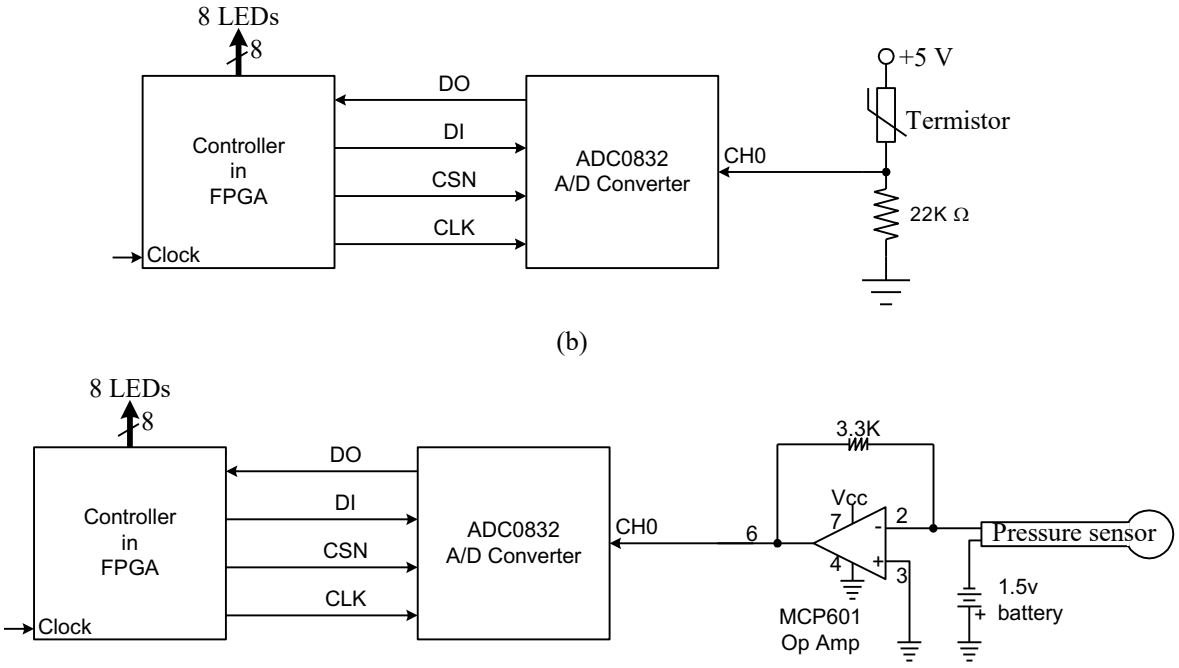


图 9.41 ADC 芯片与模拟传感器相连接：(a) 用于测试的电位计；(b) 热敏电阻式温度传感器；(c) 一个采用 MCP601 运算放大器的压力传感器。

9.9 实时时钟的 I²C 总线控制器

在这一小节中我们会设计和实现用来控制实时时钟芯片的微型控制器。实时时钟芯片与使用标准 I²C 总线协议的微型控制器相连接。微型控制器作为上层连接到 I²C 总线。实时控制芯片又作为下层连接到 I²C 总线。使用 I²C 总线作为通信渠道，主体的微型控制器可以发送和接收来自从体的数据。

9.9.1 工作原理

I²C 总线作为一个简单的双向连续总线，可以支持多个主体与从体。它由两条线组成：一个序列双向数据通道（SDA）和一个序列双向时钟通道（SCL）。在 I²C 总线定义当中，定义了 100kHz 最大时钟速率的标准模式和 400kHz 最大速率的快速模式。

每一个连接到 I²C 总线项链的硬件都是可以通过一个唯一的地址来软件寻址到的。在硬件之间的主从关系一直存在着。那些通过控制总线接入来控制数据的发送和接受的叫做主体。发送数据到总线的被叫做发送器，从总线接收数据的叫做接收器。

同一个 I²C 总线可以接入多于一个的主体和多于一个的从体。但是总线每次只能被一个主体控制，而且它负责产生序列时钟（SCL）和通过启动和停止消息的传递控制总线接入。

我们的系统由唯一一个主体构成（FPGA 上的微型控制器）和一个从体（Maxim DS3232 实时时钟芯片）如图 9.42 所示。连接主体和从体的 SDA 和 SCL 的线都是开路，都要用 5.6K 欧的上拉电阻接到 VCC。不管多少硬件是连接到这根总线上的，每根总线都只需要一个上拉电阻就够了。注意到主体控制器的实现不一定满足 I²C 总线协议的所有描述。所以如果你把这个用到其他的 I²C 总线连接方式上，主体的控制器也许会产生错误状态。

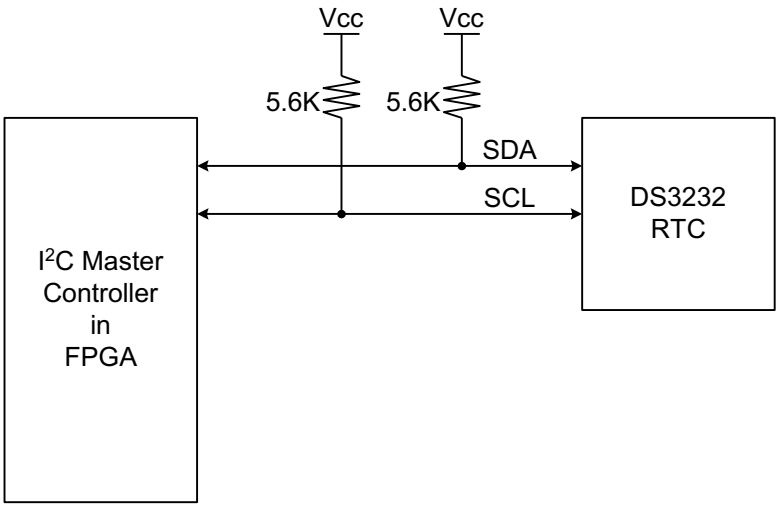


图 9.42 I²C 总线系统带有一个 FPGA 上的 I²C 主体控制器和一个从体的实时时钟芯片

当 SCL 和 SDA 线都是 1 的时候，I²C 总线是空闲状态。主体进入 START 状态（SDA 线从高变低同时 SCL 保持高电平如图 9.43(a)），启动数据传输。进入 START 状态后，总线被认为是在忙碌状态的。在 START 状态之后，主体发送一个从体地址给总线。这个地址是 7 个 bit，后面加上 1 个 bit 用来表示数据的方向（R/W）。其中 0 代表主体向从体写，1 代表主体向从体读。控制 SCL 线的主体，会在 SDA 线上发送这些 bits，每 1 个 SCL 线的时钟周期发送 1 个 bit。SDA 的线的值只能在 SCL 线为低电平的时候被改变。

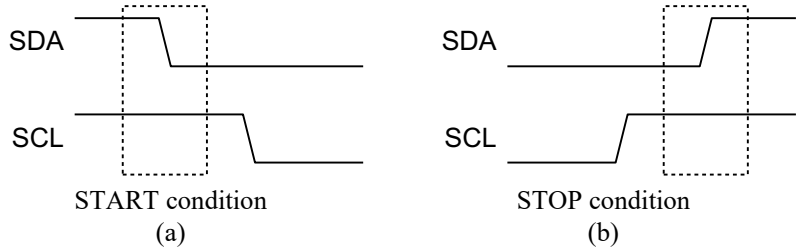


图 9.43 START(a)和 STOP(b)条件同时被主体启动。当 SDA 从高变低而 SCL 维持高电平时，START 条件发生。当 SDA 从低到高而 SCL 维持高电平时，STOP 条件发生。只有两种条件中 SDA 线可以改变电平，而且要是 SCL 为高电平时候才可以。

被寻址的那个从体硬件则在 SDA 线上发送一个确认 bit。具体方法是第 9 个 SCL 线的时钟周期中将 SDA 线拉低如图 9.44 所示。方向 bit（R / W）决定了是主体还是从体是下一个序列数据的发送者。

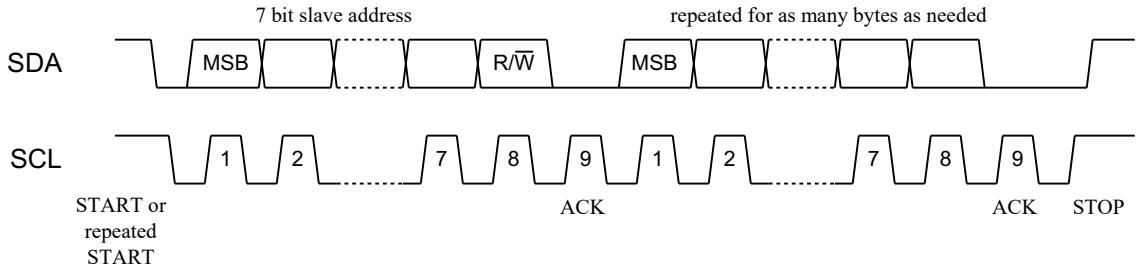


图 9.44 主体通过发送 START 条件启动数据传输。对于每 8 个 bit 的传输数据，接收器在第 9 个时钟通过拉低 SDA 线发送一个确认信号。唯一的例外是当主体接收器想要终止数据传输的时候，主体（同时也是接收者）不会发送确认信号。数据停止时通过主体发送 STOP 条件来实现的。

每个 SDA 线上发送的字节都是 8bit 长的。除了 START 和 STOP 条件，当 SCL 线是高电平的时候，SDA 线不能改变。能够传输的字节数没有被限制。每个字节必须带一个确认 bit。主体来产生确认 bit 的时钟脉冲。发送器在确认时钟脉冲内拉低 SDA 线来对接收到的字节产生确认。唯一的例外是当主体-接收器参与在传输中。这种情况下，主体-接收器必须告诉从体-发送者数据传输的结束，通过不产生确认信号。

为了告知数据传输的结束，主体需要当 SCL 线为高电平的时候，通过拉高 SDA 线来发送一个 STOP 条件，如图 9.43(b)所示。或者不发送 STOP 条件，取而代之的是主体重复发送 START 条件-，这样他可以在不用放弃接入总线的情况下改变数据传输的方向。

图 9.45(a)显示了主体向从体写 1 个字节的数据的情况（i.e.主体是发送者，从体是接收者）。主体启动数据传输，发送一个 START 条件，紧跟着 7bit 的从体地址外加一个 write(0)bit。在收到来自从体的确认后，主体发送一个寄存器序号让从体知道接下来的数据应该写入哪个寄存器。然后从体回复一个确认。主体然后发送数据字节到从体。在从体确认数据接受完后，主体发送一个 STOP 条件。

图 9.45(b)显示了主体从从体读取一个字节的数据（i.e. 主体是接收器，从体是发送器）。主体启动数据传输，发送一个 START 条件，紧跟着 7bit 的从体地址外加一个 write (0)bit。尽管主体想要接收一个字节，它首先需要发送寄存器的地址让从体知道主体可以从哪个寄存器中读取数据。在接收到来自从体的确认后，主体发送一个寄存器序号让从体知道从哪个寄存器去读取。从体然后再次发送确认信号。这一次主体需要发送一个重复的 START 条件因为它需要改变数据方向从写到读。这个重复的 START 信号紧跟着一个从体的地址，但这次后面跟着 read (1) 信号。这个从体确认然后发送确认字节给主体。这一次，因为主体是接收器，主体必须发送确认信号。如果主体想要接受更多的字节，它发送一个 0 来确认它。如果主体不想接受更多字节，它就不发任何东西，然后拉高 SDA。最后，主体发送 STOP 条件。

总结主体发送和主体接收的两种情况，在两种情况下，主体线发送一个 7bit 的从体地址和一个 write bit，紧接着是寄存器序列。在主体-发送器 情况(a)下，因为数据方向依旧是『写』，所以主体可以立刻发送一个数据字节。在主体-接收器情况(b)下，主体必须发送重复的 START 信号，然后反复发送从体地址和『写』bit，目的是改变数据传送的方向从写到读。然后，主体可以从从体的给定寄存器中接收数据字节。

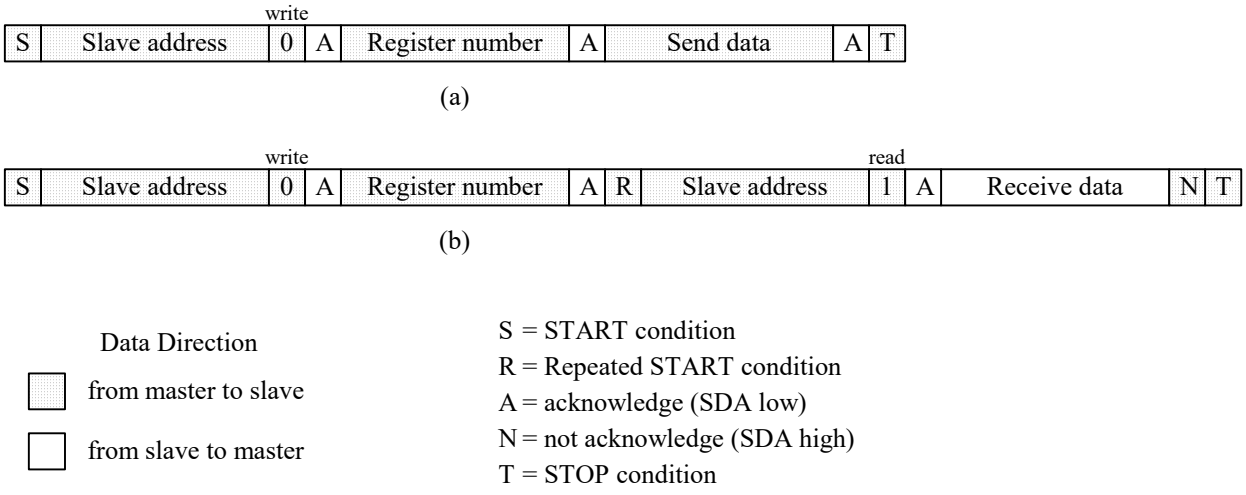


图 9.45 (a)主体发送一个字节的数据给从体 (b) 主体接收一个字节的数据来自从体 。

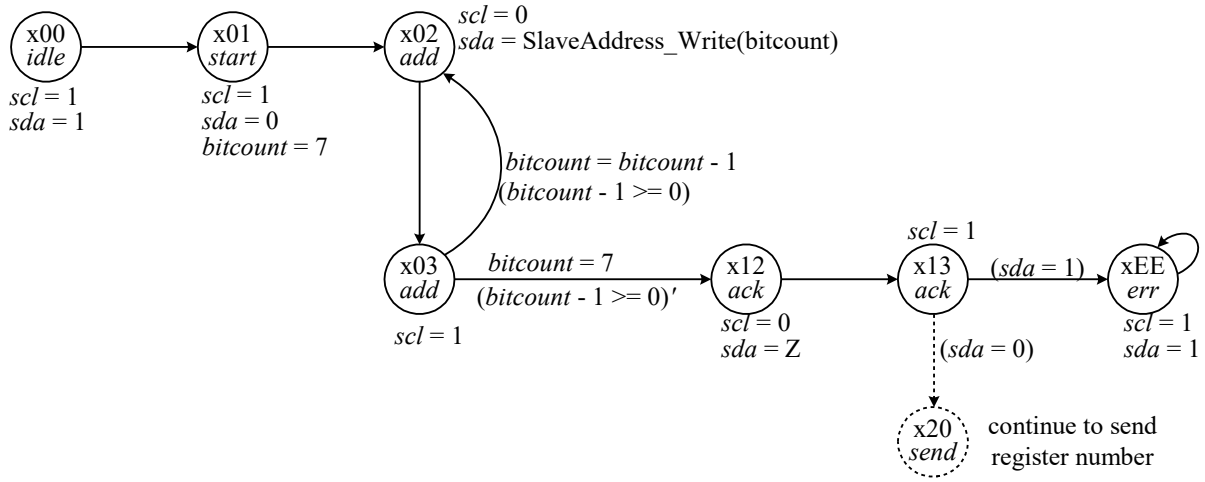
9.9.2 控制器设计

根据上述的 I²C 协议规范，我们可以得出 I²C 主体控制器的状态图。这个状态图会需要很多状态，但是其中的很多都是相同的，而且符合同样地模式。图 9.46(a) 显示了状态图最初的部分。从状态 x00 开始，FSM 转移到状态 x01 发送 Start 条件，紧跟着 7bit 的从体地址和一个 write bit。从体地址和 write bit 在之前存到了 8bit 的 SlaveAddress_Write 阵列中，然后在状态 x02 和 x03 的状态时一次一个 bit 地被移到 sda 线上。这里需要两个状态是因为 scl 线必须在 0 和 1 两个状态之间切换。在发送这 8 个 bit 之后，FSM 设置 sda 线为高

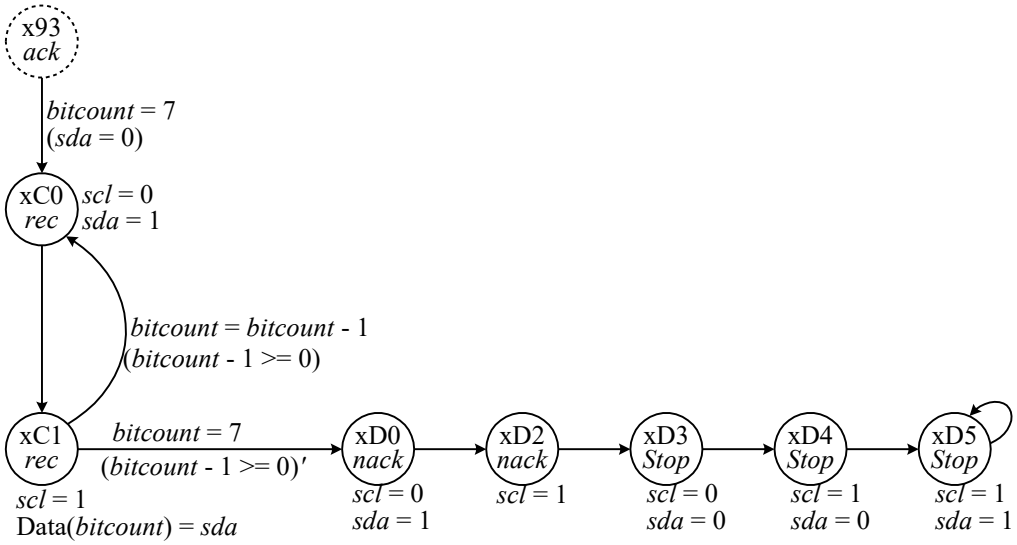
组态 Z 在状态 x12 时，来接收处于 x13 状态下的接收器发来的确认信号。如果确认信号无效(1) 那么 FSM 转移到错误状态 xEE 并且终止在那里。否则，FSM 继续转移到状态 x20 去发送寄存器序列给从体。

发送寄存器序列给从体的状态对于发送从体地址的状态 x02 和 x03 是相同的。唯一的不同在于，置入不同的 bit 值到 SDA 线当中；在这种情况下是寄存器序列而不是从体地址。

图 9.46(b)显示了从从体接收字节然后发送 Stop 条件的部分状态图。在状态 x93 中从从体接收完确认信号之后，FSM 通过设置 *bitcount* 为 7 来准备接收 8bit 的数据。FSM 在 xC0 和 xC1 之间循环 8 次之后接收了 8bit 的数据，每次讲 *sda* 上的数据接收到数据阵列当中。在接收完数据后，FSM 在 xD0 和 xD2 状态中发送一个不确认的信号后面紧跟着最后三个状态的 Stop 条件。



(a)



(b)

图 9.46 I²C 的主体控制器的部分状态图 (a)发送开始条件和从体地址 (b) 接收从体发来的数据和发送停止条件

我们利用 FSMD 模型写 HDL 代码来表示 I²C 的主体控制器。图 9.47(a)和(b)的 VHDL 代码分别对应了中的对应图 9.46 的两个状态图。整个源代码，Verilog 和 VHDL 的格式可以从书的官网上下载到。

```
output: PROCESS(CLK_200k_Hz, Go)
```

```
BEGIN
  IF(Go = '0') THEN
    -- when idle, both SDA and SCL = 1
    SCL <= '1';
    SDA01 <= '1';
    Ready <= '0';
    state <= x"00";
  ELSIF(CLK_200k_Hz'EVENT and CLK_200k_Hz = '1') THEN
    CASE state IS
    WHEN x"00" => -- Idle
      -- when idle, both SDA and SCL = 1
      SCL <= '1';   -- SCL = 1
      SDA01 <= '1'; -- SDA = 1
      Ready <= '0';
      state <= x"01";
    -- send start condition and slave address
    WHEN x"01" => -- Start
      SCL <= '1';   -- SCL stays at 1 while
      SDA01 <= '0'; -- SDA changes from 1 to 0
      Ready <= '0';
      bitcount <= 7;   -- starting bit count
      state <= x"02";
    -- send 7-bit slave address followed by R/W' bit, MSB first
    WHEN x"02" =>
      SCL <= '0';
      SDA01 <= SlaveAddress_Write(bitcount);
      state <= x"03";
    WHEN x"03" =>
      SCL <= '1';
      IF (bitcount - 1) >= 0 THEN
        bitcount <= bitcount - 1;
        state <= x"02";
      ELSE
        bitcount <= 7;
        state <= x"12";
      END IF;
    -- get acknowledgment' from slave
    WHEN x"12" =>
      SCL <= '0';
      SDA01 <= '1';
      state <= x"13";
    WHEN x"13" =>
      SCL <= '1';
      IF SDA = '1' THEN
        state <= x"EE";   -- acknowledge error
      ELSE
        state <= x"20";   -- send register address
      END IF;

    -- send 8-bit register address to slave
    WHEN x"20" =>
```

(a)

```
    -- get acknowledge signal from slave
    WHEN x"93" =>
```

```
      SCL <= '1';
      IF SDA = '1' THEN
        state <= x"EE";   -- acknowledge error
      ELSE
        bitcount <= 7;
        state <= x"C0";   -- go to receive data byte
      END IF;
    -- receive byte from RTC slave
    WHEN x"C0" =>
      SCL <= '0';
      SDA01 <= '1';
      state <= x"C1";
    WHEN x"C1" =>
      SCL <= '1';
      Data(bitcount) <= SDA; -- MSB of data read in
      IF (bitcount - 1) >= 0 THEN
        bitcount <= bitcount - 1;
        state <= x"C0";
      ELSE
        bitcount <= 7;
        state <= x"D0";
      END IF;
    WHEN x"D0" =>
      SCL <= '0';
      SDA01 <= '1'; -- send a not acknowledge' (1) signal
      state <= x"D2";
      DataOut <= Data;
    -- send a not acknowledge' (1) signal
    WHEN x"D2" =>
      SCL <= '1';
      state <= x"D3";
    -- send stop condition
    -- SDA goes from 0 to 1 while SCL is 1
    WHEN x"D3" =>
      SCL <= '0';
      SDA01 <= '0'; -- SDA starts at 0 to prepare for the 0 to 1 transition
      state <= x"D4";
    WHEN x"D4" =>
      SCL <= '1';   -- SCL goes to 1
      SDA01 <= '0'; -- SDA starts from 0
      state <= x"D5";
    WHEN x"D5" =>
      SCL <= '1';   -- SCL stays at 1 while
      SDA01 <= '1'; -- SDA goes to 1
      Ready <= '1';
      state <= x"D5";
```

图 9.47 图 9.46 中的状态图对应的两个代码: (a) 发送开始条件和从体地址 (b) 接收从体字节和发送停止信号

对 sda 信号线的赋值上需要一些特殊的注意。你可能注意到在代码中，对于所有的赋值，它用了信号名为 SDA01。SDA 信号线在 I^2C 总线中是双向的，所以它能够同时输入或者输出。其次，在 I^2C 总线中 SDA 线是开路的，所以需要一个 5.6k 欧的上拉电阻。所以为了输出逻辑 1，你需要将它设成高阻抗。为了得到高阻抗，你需要利用一个三台输出，然后给它赋值为 Z。从 SDA01 信号到 SDA 信号的真正转换发生在下述语句当中。

这句话赋值给 SDA 信号线了高阻态 Z，当内信号 SDA01 是逻辑 1 的时候。否则，SDA 信号线为逻辑 0。所以为了给 SDA 信号赋值 0 或者 1 的时候，代码赋值 0 或 1 给内部信号 SDA01，然后 SDA 信号就被置 0 或者 Z 了。

I²C 总线协议确定了标准模式下的最大的时钟速率是 100kMz。所以，我们需要确保 *scl* 信号达到这个速率。*scl* 信号由 FSM 通过不停地在两个状态下来回产生：第一个状态设置 *scl* 为 0，第二个状态设置 *scl* 为 1. 所以，对于每一个 *scl* 周期，FSM 必须经过这两个状态。为了得到 100kHz 的时钟速率，FSM 时钟速率必须快过或者等于 200kMz。如果主时钟速率是 50MHz，那么我们需要一个时钟分频器，计算 50MHz 的周期循环次数，从 0 到 250（i.e.,50,000,000 / 200,000）。为了得到 50%的占空比的 200kHz 时钟，时钟信号 CLK_200k_Hz 每 125 次触发一次(i.e., 250/2)。

9.9.3 应用实现

既然我们有了 I²C 主体控制器能够与实时时钟从体通信，我们需要一个用户界面，能够允许我们来设置实时时钟芯片的时间并将它显示在 LED 上。界面端口的顶层模块如图 9.48 所示。

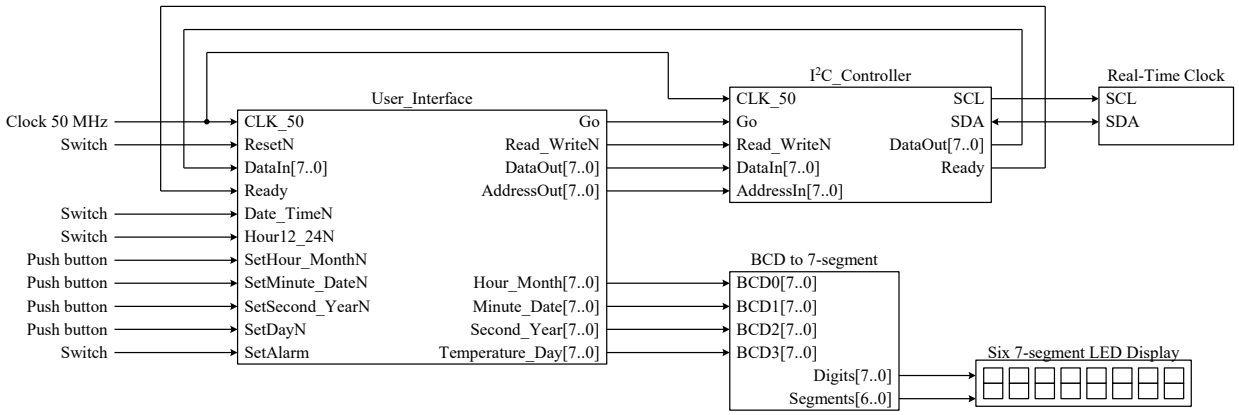


图 9.48 实时时钟的顶层模块

用户端口 FSM 有两个功能：1) 不停地向 I²C 控制器发送指令来从试试时钟芯片中读取日期和时间信号。时间和日期信息然后可以在六个 LED 上显示；2) 它等待用户开关或者按钮的输入信号。当一个开关或者按钮被处罚，FSM 通过四个控制信号线发送合适的指令给 I²C 控制器：*Go, Read_WriteN, DataIn[7..0]*和 *Address[7..0]*，来设置实时时钟芯片中相应的日期和时间寄存器。四个输出信号，*Hour_Month*，*Minute_Day, Second_Year* 和 *Temperature_Day*，会输出合适的值给 BCD 到 7 段解码器来解出正确的日期和时间信号。

用户输入包括 *Data_TimeN* 开关来选择设置日期还是时间，*Hour12_24N* 选择时间是 12 小时制还是 24 小时制，*SetHour_MonthN, SetMinute_DateN, SetSecond_YearN, SetDayN* 和 *SetAlarm* 按钮和开关来设置日期和时间信息。

9.10 USB 的 SPI 总线控制器

通用串行总线（USB）成为不同的计算机和设备彼此连接的常用方式，并且在许多情况下，取代了 RS-232 串行连接。该 USB 使得外围设备连接到计算机变得更容易和更方便用户的使用，但为了保证连接的稳健性，该协议规范做得比为 RS-232 要复杂得多。正因为如此，用 USB 连接工作相对更加复杂与困难。然而，有许多带有内部串行接口引擎（SIE）的特殊的 IC，来处理底层 USB 协议细节。

有一种这样特殊的 IC 叫做 MAX3421E USB 外设/主机控制器。它包含了所有必需的数字逻辑和必要的模拟电路。它可以实现全速 USB 主机或者外设传输，并兼容 USB 规范 2.0 版本。 MAX3421E 通过标准串行外设接口（SPI）的四根线与任何微控制器相连接，然后使用一组寄存器进行操作。 精确时间的 USB 的操作都在 MAX3421E 内部完成，使得 SPI 主体并不需要处理精确的 USB 时序要求。

9.10.1 操作原理 – SPI

为了与 MAX3421E 工作，我们需要使用 SPI 连接来对微控制器进行对接接口。 SPI 是微控制器常用的协议，用于在短距离上快速地与一个或多个外围设备进行通信。这是一个主机与从机通信的同步串行协议。主机对所有设备用三条常规的线，并对每一个设备使用一根特殊的线连接。这四个连接线分别是：

- MOSI (Master Out Slave In) – 向从机传输数据的主线
- MISO (Master In Slave Out) – 向主机传输数据的从线
- SCLK (Serial Clock) – 主机用于同步数据传输的同步信号
- SS (Slave Select) – 低电平有效的使能线，主机从而可以控制从机的开闭

该 SPI 标准是不严格的，每个不同的设备对它的实现略有不同。基本上有四种模式的传输，它们控制数据是在上升沿还是下降沿（称为时钟相位）变化，以及时钟空闲时是高电平或低电平（称为时钟极性）。MAX3421E 的使用模式 0，意味着 MOSI 和 MISO 线上的数据在下降 SCK 沿变化，并且 SCK 时钟空闲时为低电平。此外，MAX3421E 在串行线上首先发送最高显著位（MSB），并可以在速度高达 26MHz 时工作。

SPI 主机通过读取和写入芯片内部寄存器访问 MAX3421E。一个典型的数据传输包括写入第一个命令字节，用于设置寄存器地址和数据方向，然后传输附加字节读取或写入数据到寄存器或内部 FIFO 中。第一个命令字节的格式如图 9.49。前五位用于确定要访问的寄存器的地址， R'/ W 位确定到底是读取还是写入数据—0 为读，1 为写。

Reg4	Reg3	Reg2	Reg1	Reg0	0	R'/W	0

图 2.1 第一个命令字节的格式

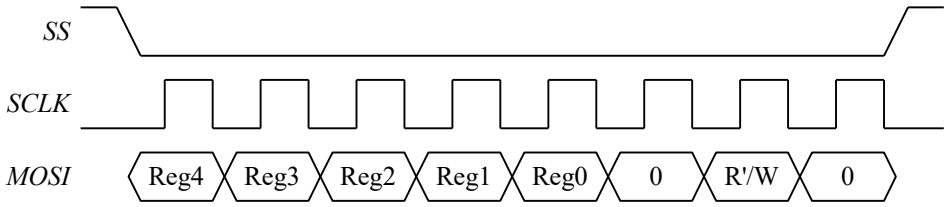


图 2.2 通过 SPI 总线传输一个字节的时序图。

9.10.2 操作原理 – USB

USB 协议比 RS-232 协议复杂得多。然而，低级详细的实现 USB 协议由 MAX3421E 的内部组装串行接口引擎（SIE）执行。 这包括

- USB 包 PID 侦查和检错
- CRC 检错和生成
- 在错误的情况下自动重试
- USB 包生成
- NRZI 数据编码与译码
- 位填充和拆箱
- USB 错误检测
- USB 总线复位，挂起和唤醒检测
- USB 暂停/恢复信号
- 自动流量控制(NAK).

该 MAX3421E 还具有一个内部振荡器和锁相环（PLL），它可以处理 USB 的时间严格的操作。为了使用 MAX3421E 实现一个 USB 主机控制器，我们只需遵循和执行常规的 USB 主机操作，通过设置外围设备地址，端点，请求类型，请求和发送包等操作。这是通过设置 MAX3421E 的内部相应的寄存器来完成的。

发送数据包后，主机控制器或通过中断或通过轮询来等待完成信号，然后检查传输结果位，以确定 USB 外设如何回应。如果数据从外围设备传来，主机控制器将在数据读出。

USB 通信的前几个步骤被称为枚举。这是当 USB 主机

9.11 习题

9.1. 尝试不同的时钟速度生成三位数的 7 段 LED 显示控制器，看看它的效果。

9.2. 写一个 HDL 程序，将 8 位无符号二进制数转换为相应的十进制数，并将十进制数在三位的 7 段 LED 显示。

9.3. 实现为解决如图 9.9（a）所示的求和输入数字的问题而设计的不正确的状态图的控制电路，来证明回车按钮不能正常工作。

9.4. 实现为解决如图 9.9（b）所示的求和输入数字的问题而设计的不正确的状态图的控制电路，来证明回车按钮不能正常工作。

9.5. 实现为解决如图 9.9（10）所示的求和输入数字的问题而设计的正确的状态图的控制电路，来证明回车按钮可以正常工作。

9.6. 设计并实现一个微控制器，将 3×4 键盘控制器与三位七段组合 LED 显示控制器相组合。最终实现当键盘上数字键被按下时，对应的数字将显示在 7 段的 LED 显示屏上。

9.7. 设计并实现一个微控制器，将 3×4 键盘控制器与三位七段组合 LED 显示控制器相组合。最终实现当可以在键盘上输入任意的最多 3 位的十进制数，对应的数字将显示在 7 段的 LED 显示屏上。使用#键作为回车键。

9.8. 设计并实现一个密码锁，用户必须输入正确的三位代码才能打开它。使用 3×4 键盘作为输入。正确的代码为 427。使用 LED 来显示锁的状态。当锁被打开时，打开 LED。锁锁上时，熄灭 LED。

9.9. 设计并实现了一个门禁报警关断控制器。一个通 - 断开关被安装在门上。当门被打开时，开关发送信号 1。此人现在有 30 秒的时间到一个 3×4 键盘上输入正确的 3 位代码。如果在 30 秒内输入正确的代码，报警将被禁止，否则，会发出报警声。使用 LED 来显示报警的状态，使用另一个 LED，来显示警报器是否被打开。

9.10. 用 VHDL 语言来实现一个 PS2 鼠标控制器并看看它是否能工作。

9.11. 手动设计一个控制电路来控制 PS2 鼠标。在 FPGA 开发板上实现它并保证它能够正常工作过。

9.12. 用一个 50MHz 的时钟来驱动 16×2 LCD FSM 控制器并看看它如何显示。能够让 LCD 正常显示的驱动 LCD 控制器能够用的最快的时钟是多少？

9.13. 使用 4 位数据格式替代 8 位数据格式来修改 16×2 LCD FSM 控制器的行为级 HDL 代码。

9.14. 实现一个微控制器来输入从 PS2 键盘中输入字符并显示在 LCD 显示屏上。

9.15. 实现一个 VGA 显示器控制器看它如何工作。

9.16. 修改 VGA 控制器来驱动一个 1024 列× 768 行分辨率的 VGA 显示器。

9.17. 设计并实现一个微控制器来从 PS2 键盘中输入字符并显示在 VGA 监控上。

9.18. 实现一个 A / D 转换器，看看它的工作原理。

9.19. 实现一个实时时钟的 I^2C 控制器，看看它的工作原理。